

Code walkthroughs behind the edit maps

Use this volume after choosing an edit path. Each section prints an existing implementation or contract linked from an edit step, its complete captured declaration/body, and the indexed call and control structure. Proposed additions remain editing plans; their source links show existing patterns.

Snapshot: 2026-10-04T23:16:35-05:00. 46 deduplicated symbols and 3 source-only anchor contexts selected from 10 edit maps; 5856 symbols are indexed in the complete explorer. This snapshot is generated only on explicit request and may lag current code. The collector parses syntax; it does not configure or execute the application build. Complete means the source and structure captured by this scanner, not complete runtime behavior. Compiler/configuration anchors are covered by the compiler reference. References absent from the captured model remain review items in the edit paths; they cannot supply a source listing here.

[Up to graphical edit paths](#) | [Interactive symbol directory](#) | [Compiler reference](#)

Print limits apply only to broad call relationships: at most 8 unique incoming callers, 8 outgoing name/analysis groups, and 3 candidate targets per outgoing group, each with exact omission counts. Signatures, selected symbol source, edit-step rationale, indexed members, and structural outlines are not truncated.

Implementation / contract	Captured source	Linked edit steps
foundation::ui::ViewDefinition	gui/shared/view_definition.hpp:9	widget / N1 widget-placement / N2 widget-placement / N4 tabs / N2 tabs / N8
foundation::ui::Application::handle	gui/shared/application.cpp:74	widget / N2 widget / N6 widget / N9 widget-placement / N6 tabs / N5 tabs / N8 core-operation / N2 core-operation / N3 core-operation / N6 commands / N3 commands / N4 commands / N6 background-task / N3 import-export / N1 import-export / N2 import-export / N4 import-export / N5 widget-runtime / N3 widget-runtime / N5
foundation::ui::Application::append_entry	gui/shared/application.cpp:49	widget / N3 widget / N4 widget / N6 widget / N7 widget / N8 validation / N6 core-operation / N3 core-operation / N5 commands / N2 widget-runtime / N4
foundation::ui::Application	gui/shared/application.hpp:12	widget / N3 widget / N7 source-files / N7 core-operation / N2 commands / N2
foundation::Store	include/foundation/store.hpp:22	widget / N4 widget / N8 source-files / N2 validation / N1 validation / N2 validation / N5 core-operation / N1 core-operation / N4 import-export / N1
foundation::ui::Application::Application	gui/shared/application.cpp:12	widget / N5 tabs / N1 validation / N5 commands / N1
foundation::host::Session::Session	gui/host/contract.hpp:45	widget / N5 widget-runtime / N2

Implementation / contract	Captured source	Linked edit steps
foundation::Store::add	src/store.cpp:33	widget / N6 widget / N8 validation / N4 widget-runtime / N8
foundation::ui::Application::publish	gui/shared/application.cpp:300	widget / N9 widget-placement / N2 widget-placement / N3 widget-placement / N5 tabs / N4 tabs / N6 validation / N6 core-operation / N6 commands / N6 background-task / N6 import-export / N6 widget-runtime / N5
foundation::ui::Application::add	gui/shared/application.cpp:29	widget-placement / N1 widget-placement / N3 widget-placement / N4 tabs / N3
gui::Rect	@gui-boundary/include/gui/geometry.hpp:13	widget-placement / N3 widget-placement / N5 tabs / N4
Example::publish	@gui-boundary/examples/application.hpp:150	widget-placement / N3 widget-placement / N5 tabs / N4
gui::WidgetState	@gui-boundary/include/gui/contract.hpp:93	widget-placement / N3 widget-placement / N5 tabs / N4
gui::WidgetSpec	@gui-boundary/include/gui/contract.hpp:79	widget-placement / N4 tabs / N2 tabs / N3
gui::validate_snapshot	@gui-boundary/include/gui/contract.hpp:234	widget-placement / N4 tabs / N2 tabs / N3 tabs / N7 commands / N4 commands / N5
gui::WidgetKey	@gui-boundary/include/gui/contract.hpp:19	widget-placement / N4 tabs / N2 tabs / N3 commands / N5
gui::PageEvent	@gui-boundary/include/gui/contract.hpp:159	widget-placement / N7 tabs / N5
gui::Snapshot	@gui-boundary/include/gui/contract.hpp:124	widget-placement / N7 tabs / N4
gui::Page	@gui-boundary/include/gui/contract.hpp:113	tabs / N1
Example::Example	@gui-boundary/examples/application.hpp:12	tabs / N1
Example::handle	@gui-boundary/examples/application.hpp:67	tabs / N5
gui::page_tabs	@gui-boundary/include/gui/presentation.hpp:35	tabs / N6
gui::Adapter	@gui-boundary/include/gui/contract.hpp:396	tabs / N7
_metadata	tools/rust_build.py:335	source-files / N6
foundation::Store::validate	src/store.cpp:17	validation / N1 validation / N4 widget-runtime / N7
validate_text	rust/text_validation/src/lib.rs:11	validation / N3 widget-runtime / N6
foundation_text_validate_v1	src/text_validation.cpp:14	validation / N3 widget-runtime / N6

Implementation / contract	Captured source	Linked edit steps
anonymous enum @14	src/text_validation.h:14	validation / N3
foundation::Store::update	src/store.cpp:47	validation / N4 core-operation / N1 core-operation / N4 core-operation / N5
foundation::ui::TextTask::start	gui/shared/task.hpp:25	validation / N5
anonymous namespace @12::run	src/main.cpp:13	validation / N6 core-operation / N1
foundation::ui::Application::advance_input_epoch	gui/shared/application.cpp:67	core-operation / N3 commands / N2 import-export / N4
gui::KeyBinding	@gui-boundary/include/gui/contract.hpp:119	commands / N5
gui::ShortcutKey	@gui-boundary/include/gui/contract.hpp:118	commands / N5
gui::normalize_event	@gui-boundary/include/gui/contract.hpp:371	commands / N5 commands / N6
foundation::ui::TaskUpdate	gui/shared/task.hpp:12	background-task / N1
foundation::ui::Application::complete_task	gui/shared/application.cpp:281	background-task / N1 background-task / N5
foundation::ui::TextTask::advance	gui/shared/task.hpp:39	background-task / N2
foundation::ui::TaskExecutor	gui/shared/task.hpp:58	background-task / N4
foundation::ui::Application::tick	gui/shared/application.cpp:274	background-task / N4
foundation::ui::Application::shutdown	gui/shared/application.cpp:266	background-task / N6
foundation::ui::Application::complete_service	gui/shared/application.cpp:232	import-export / N1 import-export / N3 import-export / N4 import-export / N6
foundation::host::file_detail::transfer	gui/host/file_services.hpp:66	import-export / N5
readTextFile	gui/host/file_services.mjs:20	import-export / N5
foundation::detail::validate_text	src/text_validation.cpp:32	widget-runtime / N6
foundation_text_validate_v1	rust/text_validation/src/lib.rs:28	widget-runtime / N6
Source-only anchor context	gui/shared/view_definition.hpp:22	widget / N1 widget-placement / N2 tabs / N2 core-operation / N2 commands / N4 widget-runtime / N1 widget-placement / N1
Source-only anchor context	rust/text_validation/src/lib.rs:1	source-files / N6
Source-only anchor context	include/foundation/store.hpp:12	core-operation / N4

foundation::ui::ViewDefinition

[struct](#) | [gui/shared/view_definition.hpp:9](#) | [Lines 9-17](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui

Captured signature

```
struct ViewDefinition
```

Why this implementation is on an edit path

[widget / N1: 1. Declare the widget \(EDIT\)](#)

Why: This ordered table already supplies widget creation, ordering, and shared layout. A new rendering capability or new event type would be a separate contract change.

[widget-placement / N2: Set vertical order and height \(EDIT\)](#)

Why: A row in this table is the ordinary vertical placement edit; coordinates are computed later.

[widget-placement / N4: Assign page and parent \(EDIT\)](#)

Why: Names such as entries.custom are widget IDs only; they do not establish tab membership.

[tabs / N2: 2. Declare page-owned content \(EDIT\)](#)

Why: The current row schema has no page or parent field; an ID prefix cannot select a tab.

[tabs / N8: Then add the page widgets \(JUMP\)](#)

Why: The tab path establishes navigation; each control still has a stable ID and its own behavior.

Complete captured source

[gui/shared/view_definition.hpp](#) | [Lines 9-17](#) | [SHA-256 a2753be85be375a526aafaaa7fb6e9e917215eb3e760f9b1d7c8318ab981f20e](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
9 | struct ViewDefinition {
10 |     std::string_view id;
11 |     gui::Kind kind;
12 |     double height;
13 |     std::string_view text, label, placeholder;
14 |     bool bold = false;
15 |     gui::TextWrap wrap = gui::TextWrap::none;
16 |     bool remove_extension = false;
17 | };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

7 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	std::string_view id;	L10
0: statement	gui::Kind kind;	L11
0: statement	double height;	L12
0: statement	std::string_view text, label, placeholder;	L13
0: statement	bool bold = false;	L14
0: statement	gui::TextWrap wrap = gui::TextWrap::none;	L15
0: statement	bool remove_extension = false;	L16

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::handle

method | [gui/shared/application.cpp:74](#) | Lines 74-166 | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui::Application

Captured signature

void Application::handle(gui::Event event)

Why this implementation is on an edit path

[widget / N2: 2. Route its click \(EDIT\)](#)

Why: Widget identity selects application behavior. The generic host already forwards events to handle.

[widget / N6: Trace the existing Add button \(JUMP\)](#)

Why: Use a concrete, source-linked implementation as the pattern for the proposed edit.

[widget / N9: 5. Project changed state \(EDIT\)](#)

Why: Presentation is a projection of shared state. adapter_.present receives the resulting snapshot.

[widget-placement / N6: Wire the placed control \(JUMP\)](#)

Why: Placement and click behavior are distinct edits, connected by the widget ID.

[tabs / N5: 5. Handle a page switch \(EDIT\)](#)

Why: The current handler has no PageEvent branch. Normalization already rejects unavailable pages, the active page, and page switches while a modal root is active.

[tabs / N8: Then add the page widgets \(JUMP\)](#)

Why: The tab path establishes navigation; each control still has a stable ID and its own behavior.

[core-operation / N2: 2. Add an Update action \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Widget identity and command meaning belong to shared Application. Existing application.cpp is already compiled.

[core-operation / N3: 3. Resolve the selected ID \(EDIT\)](#)

Why: A selected string is an identity, not a vector index. Returned snapshots are owning copies.

[core-operation / N6: 6. Project updated text and availability \(EDIT\)](#)

Why: UI text becomes visible through the shared snapshot; core mutation alone does not define action availability.

[commands / N3: 3. Route ChooseOption \(EDIT\)](#)

Why: The current menu dispatch checks option IDs alone; repeated IDs in different menus could otherwise share unintended meaning.

[commands / N4: 4. Route a current button \(EDIT\)](#)

Why: The retained contract requires every shortcut to target a current button, not a menu option ID.

[commands / N6: 6. Normalize and project state \(REUSE\)](#)

Why: Menu, button and shortcut behavior can converge on the same shared action, with normal stale/disabled input rejection.

[background-task / N3: 3. Capture inputs and route start/cancel \(EDIT\)](#)

Why: Later edits to live records must not mutate already captured work, and workers must not retain Application or adapter references.

[import-export / N1: 1. Separate content from transport \(DECISION\)](#)

Why: Changing a content format does not require copying application meaning into every backend.

[import-export / N2: 2. Change export serialization \(EDIT\)](#)

Why: The serialized representation, not merely the number of records, determines transfer size.

[import-export / N4: 4. Preserve bounds and freshness \(REUSE\)](#)

Why: A format parser alone does not authorize an unmatched service result or stale input.

[import-export / N5: 5. Reuse native/browser byte transport \(REUSE\)](#)

Why: Native and browser hosts own their file-selection/transfer authority while the application owns content meaning.

[widget-runtime / N3: Application::handle \(REUSE\)](#)

Why: This is the precise routing edit point for a new button ID.

[widget-runtime / N5: Application::publish \(REUSE\)](#)

Why: Edit this function when new state needs a visible projection.

Complete captured source

[gui/shared/application.cpp](#) | Lines 74-166 | SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
74 void Application::handle(gui::Event event) {
75     if (adapter_.closed()) return;
76     // Closing must remain possible even if measurement or painting fails.
77     if (std::holds_alternative<gui::CloseEvent>(event)) {
78         shutdown();
79         presentation_pending_ = false;
80         adapter_.close();
81         return;
```

```
82     }
83     if (shutdown_) return;
84     // Reject queued stale edits against authoritative state, not old pixels.
85     if (!gui::normalize_event(view_, event, [this](const gui::WidgetKey& key) {
86         return adapter_.scroll_offset(key);
87     }))) return;
88     if (const auto* resize = std::get_if<gui::ResizeEvent>(&event)) {
89         view_.client_size = resize->client_size;
90         view_.display_scale = resize->display_scale;
91     } else if (const auto* widget = std::get_if<gui::WidgetEvent>(&event)) {
92         auto& current = get(widget->target.id);
93         std::visit([&](const auto& input) {
94             using T = std::decay_t<decltype(input)>;
95             if constexpr (std::is_same_v<T, gui::EditText>) {
96                 advance_input_epoch();
97                 current.state.text = input.value;
98                 status_ = "Ready";
99                 status_error_ = false;
100             } else if constexpr (std::is_same_v<T, gui::Activate>) {
101                 if (widget->target.id == "entries.add") append_entry();
102                 else if (widget->target.id == "entries.task.start") {
103                     if (task_running_) return;
104                     std::vector<std::string> input;
105                     for (const auto& entry : entries_.snapshot()) input.push_back(entry.text);
106                     advance_input_epoch();
107                     task_progress_ = task_->start(std::move(input));
108                     task_running_ = true;
109                     task_status_ = "Processed 0 / " + std::to_string(task_progress_.total) + " bytes";
110                 } else if (widget->target.id == "entries.task.cancel") {
111                     if (!task_running_) return;
112                     advance_input_epoch();
113                     task_->cancel(); task_running_ = false; task_status_ = "Task cancelled";
114                 } else if (widget->target.id == "entries.remove") {
115                     auto& list = get("entries.list").state;
116                     if (!list.selected) return;
117                     advance_input_epoch();
118                     for (const auto& record : entries_.snapshot())
119                         if (list.selected == std::to_string(record.id)) entries_.erase(record.id);
120                     list.selected.reset();
121                     status_ = "Entry removed";
122                     status_error_ = false;
123                 }
124             } else if constexpr (std::is_same_v<T, gui::SubmitText>) {
125                 append_entry();
126             } else if constexpr (std::is_same_v<T, gui::ChooseOption>) {
```

```

127         if (input.id == "clear") {
128             advance_input_epoch();
129             for (const auto& record : entries_.snapshot()) entries_.erase(record.id);
130             get("entries.list").state.selected.reset();
131             status_ = "Entries cleared";
132             status_error_ = false;
133         } else if (input.id == "heading" || input.id == "import" || input.id == "export") {
134             if (next_service_ == std::numeric_limits<std::uint64_t>::max())
135                 throw std::overflow_error("Service identity exhausted");
136             gui::ServiceRequest request;
137             request.id = next_service_;
138             request.kind = input.id == "heading" ? gui::ServiceKind::prompt :
139                 input.id == "import" ? gui::ServiceKind::read_text : gui::ServiceKind::write_text;
140             request.title = input.id == "heading" ? "Change heading" : input.id == "import" ? "Import entries" :
141                 + "Export entries";
142             request.byte_limit = input.id == "heading" ? 80 : 65536;
143             if (input.id == "heading") request.value = get("entries.heading").state.text;
144             if (input.id == "export") {
145                 for (const auto& record : entries_.snapshot()) request.value += record.text + "\n";
146                 if (request.value.size() > request.byte_limit) {
147                     advance_input_epoch();
148                     status_ = "Export exceeds the 64 KiB limit"; status_error_ = true; return;
149                 }
150             }
151             advance_input_epoch();
152             if (!services_.enqueue(std::move(request)))
153                 throw std::logic_error("Service request rejected");
154             ++pending_services_;
155             ++next_service_;
156         } else if constexpr (std::is_same_v<T, gui::SelectRecord>) {
157             for (const auto& record : entries_.snapshot())
158                 if (input.id == std::to_string(record.id) && current.state.selected != input.id) {
159                     advance_input_epoch();
160                     current.state.selected = input.id;
161                 }
162             }, widget->input);
163     }
164     publish();
165 }
166 }

```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 55 indexed candidate call sites across 30 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::qualify (gui/shared/application.cpp:177)	L203, L210, L212, L214	ambiguous-name
foundation::ui::Application::qualify::edit (gui/shared/application.cpp:186)	L187	ambiguous-name
foundation::ui::Application::qualify::activate (gui/shared/application.cpp:191)	L192	ambiguous-name
foundation::host::Browser::Browser::anonymous lambda @14 (gui/host/browser.hpp:14)	L15	ambiguous-name
foundation::host (gui/host/contract.hpp:7)	L13	ambiguous-name
foundation::host::Session::Session::anonymous lambda @45 (gui/host/contract.hpp:45)	L46	ambiguous-name
foundation::host::finish_smoke (gui/host/qualification.hpp:12)	L13	ambiguous-name
main (gui/hosts/framebuffer_main.cpp:10)	L40	ambiguous-name

Print scope: 22 additional caller symbols containing 44 candidate call sites are omitted. Map-linked and same-file callers are listed first. All callers in explorer

Outgoing: 10 detected call sites grouped into 10 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
shutdown L78	foundation::ui::Application::shutdown (gui/shared/application.cpp:266) foundation::host::AdapterServices::shutdown (gui/host/contract.hpp:34) foundation::host::FileServices::shutdown (gui/host/file_services.hpp:164) +6 additional candidate targets omitted; all targets	ambiguous-name
publish L165	foundation::ui::Application::publish (gui/shared/application.cpp:300) Example::publish (@gui-boundary/examples/application.hpp:150) anonymous namespace @5::App::publish (gui/tests/bezel_test.cpp:39)	ambiguous-name
gui::normalize_event L85	gui::normalize_event (@gui-boundary/include/gui/contract.hpp:371)	name-candidate
adapter_.closed L75	No indexed target; external / unresolved.	unresolved
std::holds_alternative<gui::CloseEvent> L77	No indexed target; external / unresolved.	unresolved
adapter_.close L80	foundation::host::file_detail::File::close (gui/host/file_services.hpp:34) anonymous namespace @18::Context::close (tools/windows_gl_probe.cpp:143)	ambiguous-name
std::get_if<gui::ResizeEvent> L88	No indexed target; external / unresolved.	unresolved

Call name / call sites	Candidate target names / captured locations	Analysis
<code>std::get_if<gui::WidgetEvent></code> L91	No indexed target; external / unresolved.	unresolved

Print scope: 2 additional outgoing groups containing 2 call sites are omitted. Groups containing map-linked or same-file candidates are listed first. [All outgoing calls in explorer](#)

Complete indexed structural outline

25 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(adapter_.closed())	L75
1: then	then	L75
2: return	return;	L75
0: branch	(std::holds_alternative<gui::CloseEvent>(event))	L77
1: then	then	L77
2: statement	shutdown();	L78
2: statement	presentation_pending_ = false;	L79
2: statement	adapter_.close();	L80
2: return	return;	L81
0: branch	(shutdown_)	L83
1: then	then	L83
2: return	return;	L83
0: branch	(!gui::normalize_event(view_, event, [this](const gui::WidgetKey& key) { return adapter_.scroll_offset(key); }))	L85
1: then	then	L87
2: return	return;	L87
0: branch	(const auto* resize = std::get_if<gui::ResizeEvent>(&event))	L88
1: then	then	L88
2: statement	view_.client_size = resize->client_size;	L89
2: statement	view_.display_scale = resize->display_scale;	L90
1: else	else	L91
2: branch	(const auto* widget = std::get_if<gui::WidgetEvent>(&event))	L91
3: then	then	L91
4: statement	auto& current = get(widget->target.id);	L92

Depth / structure	Captured label	Source
4: statement	std::visit([&](const auto& input) { using T = std::decay_t<decltype(input)>; if constexpr (std::is_same_v<T, gui::EditText>) { advance_input_epoch(); current.state.text = input.va...	L93
0: statement	publish();	L165

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::append_entry

[method](#) | [gui/shared/application.cpp:49](#) | [Lines 49-65](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui::Application

Captured signature

void Application::append_entry()

Why this implementation is on an edit path

[widget / N3: 3. Add on_custom\(\) \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Separate event dispatch from feature behavior, so another event can reuse the same action.

[widget / N4: 4. What does the helper own? \(DECISION\)](#)

Why: This is the decision that determines the next file to edit.

[widget / N6: Trace the existing Add button \(JUMP\)](#)

Why: Use a concrete, source-linked implementation as the pattern for the proposed edit.

[widget / N7: 4a. apply_custom_action\(\) \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Feature state and command meaning belong here. Existing .cpp files are already compiled.

[widget / N8: 4b. custom_operation\(\) \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Core behavior should be usable without a GUI adapter or widget ID.

[validation / N6: 6. Follow existing consumers \(REUSE\)](#)

Why: The validation remains one reusable behavior rather than a separate rule per frontend.

[core-operation / N3: 3. Resolve the selected ID \(EDIT\)](#)

Why: A selected string is an identity, not a vector index. Returned snapshots are owning copies.

[core-operation / N5: 5. Handle rejected/missing input \(EDIT\)](#)

Why: Invalid replacement text can throw before missing-ID lookup, so both outcomes need explicit handling.

[commands / N2: 2. Implement one command helper \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: A menu action and a button/shortcut can call the same helper.

[widget-runtime / N4: Application::append_entry \(REUSE\)](#)

Why: This is the model for the function below your event handler.

Complete captured source

gui/shared/application.cpp | Lines 49-65 | SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
49 void Application::append_entry() {
50     advance_input_epoch();
51     const auto& text = get("entries.editor").state.text;
52     try {
53         // Core owns collection validation, capacity, identity and mutation.
54         entries_.add(text);
55         get("entries.editor").state.text.clear();
56         status_ = "Entry added";
57         status_error_ = false;
58     } catch (const std::invalid_argument& error) {
59         status_ = error.what();
60         status_error_ = true;
61     } catch (const std::length_error& error) {
62         status_ = error.what();
63         status_error_ = true;
64     }
65 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 2 indexed candidate call sites across 1 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::handle::anonymous lambda @93 (gui/shared/application.cpp:93)	L101, L125	name-candidate

Outgoing: 7 detected call sites grouped into 5 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
advance_input_epoch L50	foundation::ui::Application::advance_input_epoch (gui/shared/application.cpp:67)	name-candidate
entries_.add L54	foundation::ui::Application::add (gui/shared/application.cpp:29) foundation::Store::add (src/store.cpp:33) Example::add (@gui-boundary/examples/application.hpp:132) +3 additional candidate targets omitted; all targets	ambiguous-name

Call name / call sites	Candidate target names / captured locations	Analysis
<code>get</code> L51, L55	Example::get (@gui-boundary/examples/application.hpp:141) foundation::ui::Application::get (gui/shared/application.hpp:58) foundation::Store::get (src/store.cpp:65)	ambiguous-name
<code>get("entries.editor").state.text.clear</code> L55	No indexed target; external / unresolved.	unresolved
<code>error.what</code> L59, L62	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

14 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	<code>advance_input_epoch();</code>	L50
0: statement	<code>const auto& text = get("entries.editor").state.text;</code>	L51
0: try	<code>try</code>	L52
1: try_body	<code>try body</code>	L52
2: statement	<code>entries_.add(text);</code>	L54
2: statement	<code>get("entries.editor").state.text.clear();</code>	L55
2: statement	<code>status_ = "Entry added";</code>	L56
2: statement	<code>status_error_ = false;</code>	L57
1: catch	<code>catch (const std::invalid_argument& error)</code>	L58
2: statement	<code>status_ = error.what();</code>	L59
2: statement	<code>status_error_ = true;</code>	L60
1: catch	<code>catch (const std::length_error& error)</code>	L61
2: statement	<code>status_ = error.what();</code>	L62
2: statement	<code>status_error_ = true;</code>	L63

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application

class | [gui/shared/application.hpp:12](#) | [Lines 12-62](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui

Captured signature

class Application

Why this implementation is on an edit path

[widget / N3: 3. Add on_custom\(\) \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Separate event dispatch from feature behavior, so another event can reuse the same action.

[widget / N7: 4a. apply_custom_action\(\) \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Feature state and command meaning belong here. Existing .cpp files are already compiled.

[source-files / N7: Header or existing-file method \(EDIT\)](#)

Why: CMake source registration is required for a new .cpp, not merely for a new function declaration.

[core-operation / N2: 2. Add an Update action \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Widget identity and command meaning belong to shared Application. Existing application.cpp is already compiled.

[commands / N2: 2. Implement one command helper \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: A menu action and a button/shortcut can call the same helper.

Owning type: 7 indexed members in this declaration file. Members not recognized by the syntax scanner remain visible in the complete source below.

Indexed member	Kind / line	Signature
foundation::ui::Application::view	method L15	const gui::Snapshot& view() const noexcept
foundation::ui::Application::task_progress	method L21	const TaskUpdate& task_progress() const noexcept
foundation::ui::Application::task_running	method L22	bool task_running() const noexcept
foundation::ui::Application::~~Application	method L23	~Application()
foundation::ui::Application::presentation_pending	method L24	bool presentation_pending() const noexcept
foundation::ui::Application::input_epoch	method L27	std::uint64_t input_epoch() const noexcept

Indexed member	Kind / line	Signature
foundation::ui::Application::get	method L58	gui::Widget& get(std::string_view id)

Complete captured source

[gui/shared/application.hpp](#) | Lines 12-62 | SHA-256 910ad75e234603f29bad7fbd6094b209c8b682f83766ec1cdcdab2d0a0d6a87e

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```

12  class Application {
13  public:
14      explicit Application(gui::Adapter& adapter, std::unique_ptr<TaskExecutor> executor = make_task_executor());
15      const gui::Snapshot& view() const noexcept { return view_; }
16      void handle(gui::Event event);
17      void retry_presentation();
18      void tick();
19      void shutdown() noexcept;
20      bool complete_task(const TaskUpdate& update);
21      const TaskUpdate& task_progress() const noexcept { return task_progress_; }
22      bool task_running() const noexcept { return task_running_; }
23      ~Application() { shutdown(); }
24      bool presentation_pending() const noexcept { return presentation_pending_; }
25      // Authoritative input meaning, independent of painting/revision retries.
26      // UINT64_MAX is permanently invalid for retained input tokens.
27      std::uint64_t input_epoch() const noexcept { return input_epoch_; }
28      std::optional<gui::ServiceRequest> next_service();
29      bool complete_service(gui::ServiceResult result);
30
31      // Demonstrates an ordinary feature extension without modifying a renderer.
32      void enable_remove_feature();
33      // Installed qualification keeps every feature identity on the shared side.
34      // The host supplies only a bounded presentation/event-loop step.
35      void qualify(const std::function<void()>& present);
36
37  private:
38      gui::Adapter& adapter_;
39      gui::Snapshot view_;
40      gui::ServiceQueue services_;
41      static constexpr std::size_t entry_limit_ = 1000;
42      foundation::Store entries_{entry_limit_};
43      std::uint64_t next_service_ = 1;
44      std::uint64_t input_epoch_ = 1;
45      std::size_t pending_services_ = 0;
46      bool shutdown_ = false;

```

```
47     bool presentation_pending_ = false;
48     bool remove_feature_ = false;
49     std::string status_ = "Ready";
50     bool status_error_ = false;
51     std::unique_ptr<TaskExecutor> task_;
52     TaskUpdate task_progress_;
53     bool task_running_ = false;
54     std::string task_status_ = "Task ready";
55
56     gui::Widget& add(const ViewDefinition& definition);
57     static gui::Widget& lookup(gui::Snapshot& view, std::string_view id);
58     gui::Widget& get(std::string_view id) { return lookup(view_, id); }
59     void append_entry();
60     void advance_input_epoch();
61     void publish();
62 };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 1 detected call sites grouped into 1 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
make_task_executor L14	foundation::ui::make_task_executor (gui/shared/task_cooperative.cpp:3) foundation::ui::make_task_executor (gui/shared/task_native.cpp:69)	ambiguous-name

Complete indexed structural outline

34 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	public	L13
0: statement	explicit Application(gui::Adapter& adapter, std::unique_ptr<TaskExecutor> executor = make_task_executor());	L14
0: statement	void handle(gui::Event event);	L16
0: statement	void retry_presentation();	L17
0: statement	void tick();	L18
0: statement	void shutdown() noexcept;	L19
0: statement	bool complete_task(const TaskUpdate& update);	L20

Depth / structure	Captured label	Source
0: statement	std::optional<gui::ServiceRequest> next_service();	L28
0: statement	bool complete_service(gui::ServiceResult result);	L29
0: statement	void enable_remove_feature();	L32
0: statement	void qualify(const std::function<void()>& present);	L35
0: statement	private	L37
0: statement	gui::Adapter& adapter_;	L38
0: statement	gui::Snapshot view_;	L39
0: statement	gui::ServiceQueue services_;	L40
0: statement	static constexpr std::size_t entry_limit_ = 1000;	L41
0: statement	foundation::Store entries_{entry_limit_};	L42
0: statement	std::uint64_t next_service_ = 1;	L43
0: statement	std::uint64_t input_epoch_ = 1;	L44
0: statement	std::size_t pending_services_ = 0;	L45
0: statement	bool shutdown_ = false;	L46
0: statement	bool presentation_pending_ = false;	L47
0: statement	bool remove_feature_ = false;	L48
0: statement	std::string status_ = "Ready";	L49
0: statement	bool status_error_ = false;	L50
0: statement	std::unique_ptr<TaskExecutor> task_;	L51
0: statement	TaskUpdate task_progress_;	L52
0: statement	bool task_running_ = false;	L53
0: statement	std::string task_status_ = "Task ready";	L54
0: statement	gui::Widget& add(const ViewDefinition& definition);	L56
0: statement	static gui::Widget& lookup(gui::Snapshot& view, std::string_view id);	L57
0: statement	void append_entry();	L59
0: statement	void advance_input_epoch();	L60
0: statement	void publish();	L61

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::Store

[class](#) | [include/foundation/store.hpp:22](#) | [Lines 22-43](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation

Captured signature

class Store

Why this implementation is on an edit path

[widget / N4: 4. What does the helper own? \(DECISION\)](#)

Why: This is the decision that determines the next file to edit.

[widget / N8: 4b. custom_operation\(\) \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Core behavior should be usable without a GUI adapter or widget ID.

[source-files / N2: Core .cpp \(EDIT\)](#)

Why: A new file in src/ is not automatically compiled.

[validation / N1: 1. Identify the rule owner \(DECISION\)](#)

Why: CLI, GUI and import should receive the same rule through core; a UI-only filter would leave other callers different.

[validation / N2: 2. State the accepted input \(EDIT\)](#)

Why: A newcomer needs to know both what is accepted and what happens on rejection.

[validation / N5: 5. Follow mirrored limits \(EDIT\)](#)

Why: The current private text ABI is bounded to 256 bytes, and tasks separately enforce 256-byte records.

[core-operation / N1: 1. Reuse the existing operation \(DECISION\)](#)

Why: The same API can serve a future CLI command; the current CLI has no edit command or persistent store.

[core-operation / N4: 4. Call Store::update \(REUSE\)](#)

Why: update validates text first, returns false for a missing ID when the text is valid, and prepares a replacement string before swapping it into place. The record ID stays unchanged.

[import-export / N1: 1. Separate content from transport \(DECISION\)](#)

Why: Changing a content format does not require copying application meaning into every backend.

Complete captured source

[include/foundation/store.hpp](#) | [Lines 22-43](#) | [SHA-256 9fbe158ca20da19f21c0c69b79ec72a3ab559e3b91931f854b955a0c55e50e68](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
22 | class Store {
23 | public:
24 |     static constexpr std::size_t max_text_bytes = 256;
25 |     static constexpr std::size_t max_capacity = 4096;
26 |
27 |     explicit Store(std::size_t capacity = 64);
28 |     // Text is 1..256 printable ASCII bytes; duplicates are allowed.
```

```
29 // Invalid input throws invalid_argument; a full store throws length_error.
30 // A failed operation leaves the collection and next ID unchanged.
31 RecordId add(std::string_view text);
32 bool update(RecordId id, std::string_view text);
33 bool erase(RecordId id);
34 [[nodiscard]] std::optional<Record> get(RecordId id) const;
35 [[nodiscard]] std::vector<Record> snapshot() const;
36 [[nodiscard]] std::size_t size() const noexcept;
37
38 private:
39     static void validate(std::string_view text);
40     std::size_t capacity_;
41     RecordId next_id_ = 1;
42     std::vector<Record> records_;
43 };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

15 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	public	L23
0: statement	static constexpr std::size_t max_text_bytes = 256;	L24
0: statement	static constexpr std::size_t max_capacity = 4096;	L25
0: statement	explicit Store(std::size_t capacity = 64);	L27
0: statement	RecordId add(std::string_view text);	L31
0: statement	bool update(RecordId id, std::string_view text);	L32
0: statement	bool erase(RecordId id);	L33
0: statement	[[nodiscard]] std::optional<Record> get(RecordId id) const;	L34
0: statement	[[nodiscard]] std::vector<Record> snapshot() const;	L35

Depth / structure	Captured label	Source
0: statement	[[nodiscard]] std::size_t size() const noexcept;	L36
0: statement	private	L38
0: statement	static void validate(std::string_view text);	L39
0: statement	std::size_t capacity_;	L40
0: statement	RecordId next_id_ = 1;	L41
0: statement	std::vector<Record> records_;	L42

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::Application

method | [gui/shared/application.cpp:12](#) | [Lines 12-27](#) | [Interactive symbol / full call lists](#)

Owner: foundation::ui::Application

Captured signature

Application::Application(gui::Adapter& adapter, std::unique_ptr<TaskExecutor> executor) : adapter_(adapter), task_(std::move(executor))

Why this implementation is on an edit path

[widget / N5: Creation and event plumbing \(REUSE\)](#)

Why: Ordinary application widget behavior is shared across backends.

[tabs / N1: 1. Declare pages and selection \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: The tab registry and selection belong to the shared Snapshot. This app currently initializes neither.

[validation / N5: 5. Follow mirrored limits \(EDIT\)](#)

Why: The current private text ABI is bounded to 256 bytes, and tasks separately enforce 256-byte records.

[commands / N1: 1. Declare the menu option \(EDIT\)](#)

Why: Options declare user intent; generic renderers do not implement product command meaning.

Complete captured source

[gui/shared/application.cpp](#) | [Lines 12-27](#) | [SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
12 Application::Application(gui::Adapter& adapter, std::unique_ptr<TaskExecutor> executor)
13     : adapter_(adapter), task_(std::move(executor)) {
14     if (!task_) throw std::invalid_argument("Task executor is required");
15     view_.title = "Entry list";
```

```
16     for (const auto& definition : view_definition)
17         if (!definition.remove_extension) add(definition);
18     get("entries.editor").spec.text_policy = {false, false, foundation::Store::max_text_bytes, gui::SubmitKey::enter};
19     get("entries.options").state.options = {"clear", "Clear entries", "", true},
20                                           {"heading", "Change heading", "", true},
21                                           {"import", "Import entries", "", true},
22                                           {"export", "Export entries", "", true}};
23     auto& entries = get("entries.list");
24     entries.spec.row_height = 32;
25     entries.spec.follow_tail = true;
26     publish();
27 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 7 detected call sites grouped into 5 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
add L17	foundation::ui::Application::add (gui/shared/application.cpp:29) foundation::Store::add (src/store.cpp:33) Example::add (@gui-boundary/examples/application.hpp:132) +3 additional candidate targets omitted; all targets	ambiguous-name
publish L26	foundation::ui::Application::publish (gui/shared/application.cpp:300) Example::publish (@gui-boundary/examples/application.hpp:150) anonymous namespace @5::App::publish (gui/tests/bezel_test.cpp:39)	ambiguous-name
std::move L13	No indexed target; external / unresolved.	unresolved
std::invalid_argument L14	No indexed target; external / unresolved.	unresolved
get L18, L19, L23	Example::get (@gui-boundary/examples/application.hpp:141) foundation::ui::Application::get (gui/shared/application.hpp:58) foundation::Store::get (src/store.cpp:65)	ambiguous-name

Complete indexed structural outline

15 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: initialization	: adapter_(adapter), task_(std::move(executor))	L13
0: branch	(!task_)	L14
1: then	then	L14
2: throw	throw std::invalid_argument("Task executor is required");	L14
0: statement	view_.title = "Entry list";	L15
0: loop	for (const auto& definition : view_definition)	L16
1: branch	(!definition.remove_extension)	L17
2: then	then	L17
3: statement	add(definition);	L17
0: statement	get("entries.editor").spec.text_policy = {false, false, foundation::Store::max_text_bytes, gui::SubmitKey::enter};	L18
0: statement	get("entries.options").state.options = {{"clear", "Clear entries", "", true}, {"heading", "Change heading", "", true}, {"import", "Import entries", "", true}, {"export", "Export e...	L19
0: statement	auto& entries = get("entries.list");	L23
0: statement	entries.spec.row_height = 32;	L24
0: statement	entries.spec.follow_tail = true;	L25
0: statement	publish();	L26

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::host::Session::Session

method | [gui/host/contract.hpp:45](#) | Lines 45-47 | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::host::Session

Captured signature

Session() : adapter([this](const gui::Event& event) { if (current_) { current_->handle(event); services(); } }), application(adapter)

Why this implementation is on an edit path

[widget / N5: Creation and event plumbing \(REUSE\)](#)

Why: Ordinary application widget behavior is shared across backends.

[widget-runtime / N2: Generic event forwarding \(REUSE\)](#)

Why: A feature click does not need separate business handlers for each backend.

Complete captured source

gui/host/contract.hpp | Lines 45-47 | SHA-256 afd443a6ad7bb211d4dba795931c3d0d35110a7450fb37faa8004ddf7935dc8a

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
45 |         Session() : adapter([this](const gui::Event& event) {
46 |             if (current_) { current_->handle(event); services(); }
47 |         })), application(adapter) { current_ = &application; }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

2 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: initialization	: adapter([this](const gui::Event& event) { if (current_) { current_->handle(event); services(); } })), application(adapter)	L45
0: statement	current_ = &application;	L47

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::Store::add

method | src/store.cpp:33 | Lines 33-45 | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::Store

Captured signature

RecordId Store::add(std::string_view text)

Why this implementation is on an edit path

[widget / N6: Trace the existing Add button \(JUMP\)](#)

Why: Use a concrete, source-linked implementation as the pattern for the proposed edit.

[widget / N8: 4b. custom_operation\(\) \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Core behavior should be usable without a GUI adapter or widget ID.

[validation / N4: 4. Apply rules before mutation \(EDIT\)](#)
Why: Returning an old ASCII-only message for a different rule would mislead the caller.

[widget-runtime / N8: Store::add \(REUSE\)](#)
Why: Reusable domain rules are implemented here, outside GUI event routing.

Complete captured source

src/store.cpp | Lines 33-45 | SHA-256 59580ea4cdb305bba9c97c8faaccbf2c390b55893a40785272a3f7a72888a0c3

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
33 | RecordId Store::add(std::string_view text) {
34 |     validate(text);
35 |     if (records_.size() == capacity_) {
36 |         throw std::length_error("collection is full");
37 |     }
38 |     if (next_id_ == std::numeric_limits<RecordId>::max()) {
39 |         throw std::overflow_error("record identifiers exhausted");
40 |     }
41 |     const auto id = next_id_;
42 |     records_.push_back({id, std::string(text)});
43 |     ++next_id_;
44 |     return id;
45 | }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 47 indexed candidate call sites across 19 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
Example::Example (@gui-boundary/examples/application.hpp:12)	L15, L16, L17, L20, L23, L24, L25, L28, L31, L35, L36, L37, L38, L39, L42	ambiguous-name
foundation::ui::Application::Application (gui/shared/application.cpp:12)	L17	ambiguous-name
foundation::ui::Application::append_entry (gui/shared/application.cpp:49)	L54	ambiguous-name
foundation::ui::Application::complete_service (gui/shared/application.cpp:232)	L253	ambiguous-name
anonymous namespace @12::run (src/main.cpp:13)	L21, L36	ambiguous-name
foundation::host::Bezel::capture (gui/host/bezel.hpp:124)	L155, L156, L158, L160	ambiguous-name
foundation::ui::Application::enable_remove_feature (gui/shared/application.cpp:168)	L172	ambiguous-name
main (examples/consumer/main.cpp:12)	L17	ambiguous-name

Print scope: 11 additional caller symbols containing 21 candidate call sites are omitted. Map-linked and same-file callers are listed first. [All callers in explorer](#)
Outgoing: 7 detected call sites grouped into 7 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
validate L34	foundation::Store::validate (src/store.cpp:17)	name-candidate
records_.size L35	foundation::Store::size (src/store.cpp:73)	name-candidate
std::length_error L36	No indexed target; external / unresolved.	unresolved
std::numeric_limits<RecordId>::max L38	No indexed target; external / unresolved.	unresolved
std::overflow_error L39	No indexed target; external / unresolved.	unresolved
records_.push_back L42	No indexed target; external / unresolved.	unresolved
std::string L42	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

11 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	<code>validate(text);</code>	L34
0: branch	<code>(records_.size() == capacity_)</code>	L35
1: then	<code>then</code>	L35
2: throw	<code>throw std::length_error("collection is full");</code>	L36
0: branch	<code>(next_id_ == std::numeric_limits<RecordId>::max())</code>	L38
1: then	<code>then</code>	L38
2: throw	<code>throw std::overflow_error("record identifiers exhausted");</code>	L39
0: statement	<code>const auto id = next_id_;</code>	L41
0: statement	<code>records_.push_back({id, std::string(text)});</code>	L42
0: statement	<code>++next_id_;</code>	L43
0: return	<code>return id;</code>	L44

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::publish

[method](#) | [gui/shared/application.cpp:300](#) | [Lines 300-360](#) | [Interactive symbol](#) / [full call lists](#)

Owner: `foundation::ui::Application`

Captured signature

```
void Application::publish()
```

Why this implementation is on an edit path

[widget / N9: 5. Project changed state \(EDIT\)](#)

Why: Presentation is a projection of shared state. `adapter_.present` receives the resulting snapshot.

[widget-placement / N2: Set vertical order and height \(EDIT\)](#)

Why: A row in this table is the ordinary vertical placement edit; coordinates are computed later.

[widget-placement / N3: Change grouping or spacing \(EDIT\)](#)

Why: The layout tree computes bounds; `WidgetSpec.parent` supplies containment and clipping. Both must agree.

[widget-placement / N5: Let shared layout place it \(REUSE\)](#)

Why: Changing a backend-specific pixel position would bypass the shared placement source.

[tabs / N4: 4. Lay out content and tab bar \(EDIT\)](#)

Why: Declaring Page entries alone does not allocate visible tab-bar space or place their contents.

[tabs / N6: 6. Reuse tab rendering \(REUSE\)](#)

Why: Pages are an existing GUI contract capability. Supplier excerpts are evidence and examples, not edit targets.

[validation / N6: 6. Follow existing consumers \(REUSE\)](#)

Why: The validation remains one reusable behavior rather than a separate rule per frontend.

[core-operation / N6: 6. Project updated text and availability \(EDIT\)](#)

Why: UI text becomes visible through the shared snapshot; core mutation alone does not define action availability.

[commands / N6: 6. Normalize and project state \(REUSE\)](#)

Why: Menu, button and shortcut behavior can converge on the same shared action, with normal stale/disabled input rejection.

[background-task / N6: 6. Project controls and close owned work \(REUSE\)](#)

Why: Responsiveness includes cancellation, late update rejection and complete owned-work cleanup, not just showing progress.

[import-export / N6: 6. Project honest completion state \(EDIT\)](#)

Why: The visible result must distinguish a successfully parsed replacement, rejected content and host handoff.

[widget-runtime / N5: Application::publish \(REUSE\)](#)

Why: Edit this function when new state needs a visible projection.

Complete captured source

[gui/shared/application.cpp](#) | [Lines 300-360](#) | [SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
300 void Application::publish() {
301     presentation_pending_ = true;
302     auto next = view_;
303     auto& list = lookup(next, "entries.list").state;
304     list.records.clear();
305     for (const auto& entry : entries_.snapshot()) {
306         gui::Record record;
307         record.id = std::to_string(entry.id);
308         record.accessible_text = entry.text;
309         record.cells = {{entry.text, {}, {}}};
310         list.records.push_back(std::move(record));
311     }
312     if (list.selected && std::none_of(list.records.begin(), list.records.end(),
313         [&](const gui::Record& row) { return list.selected == row.id; })) list.selected.reset();
314     lookup(next, "entries.count").state.text = std::to_string(list.records.size()) + " entries";
315     lookup(next, "entries.status").state.text = status_;
316     lookup(next, "entries.status").state.font.tone = status_error_ ? gui::Tone::error : gui::Tone::muted;
317     gui::LayoutNode panel;
318     panel.id = "entries.form";
319     panel.kind = gui::LayoutKind::column;
320     panel.padding = {8, 8, 8, 8};
321     panel.gap = 4;
322     const auto child = [&](const std::string& id, double height) {
323         gui::LayoutNode node;
324         node.id = id;
325         node.height = height;
326         panel.children.push_back(std::move(node));
327     };
328     for (const auto& definition : view_definition)
329         if (definition.id != panel.id && (!definition.remove_extension || remove_feature_))
330             child(std::string(definition.id), definition.height);
331     const auto width = std::max(0.0, next.client_size.width - 32);
332     const auto height = std::max(0.0, next.client_size.height - 32);
333     const auto boxes = gui::compose_layout(panel, {16, 16, width, height},
334         [&](std::string_view id, double available) {
335             const auto& widget = lookup(next, id);
336             if (widget.spec.kind != gui::Kind::label) return gui::Size{available, 0};
337             return adapter_.measure_text({widget.state.text, widget.state.font,
338                 available, next.display_scale, widget.state.wrap});
339         });
340     for (const auto& box : gui::flatten_layout(boxes))
341         lookup(next, box.id).state.bounds = box.bounds;
342     auto& group = lookup(next, "entries.form").state;
343     group.content_size = {boxes.content.width, boxes.content.height};
344     group.bounds.height = std::min(height, boxes.bounds.height);
```

```
345     group.content_clip = gui::Rect{8, 8, std::max(0.0, width - 16),
346                               std::max(0.0, group.bounds.height - 16)};
347     list.content_size.width = list.bounds.width;
348     for (auto& record : list.records)
349         record.cells.front().bounds = {4, 0, std::max(0.0, list.bounds.width - 8), 32};
350     lookup(next, "entries.add").state.enabled =
351         !lookup(next, "entries.editor").state.text.empty() && list.records.size() < entry_limit_;
352     if (remove_feature_) lookup(next, "entries.remove").state.enabled = list.selected.has_value();
353     lookup(next, "entries.task.start").state.enabled = !task_running_;
354     lookup(next, "entries.task.cancel").state.enabled = task_running_;
355     lookup(next, "entries.task.status").state.text = task_status_;
356     ++next.revision;
357     adapter_.present(next);
358     view_ = std::move(next);
359     presentation_pending_ = false;
360 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 23 indexed candidate call sites across 20 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::Application (gui/shared/application.cpp:12)	L26	ambiguous-name
foundation::ui::Application::handle (gui/shared/application.cpp:74)	L165	ambiguous-name
foundation::ui::Application::complete_service (gui/shared/application.cpp:232)	L262	ambiguous-name
foundation::ui::Application::complete_task (gui/shared/application.cpp:281)	L292	ambiguous-name
Example::Example (@gui-boundary/examples/application.hpp:12)	L44	ambiguous-name
Example::handle (@gui-boundary/examples/application.hpp:67)	L113, L119, L121	ambiguous-name
foundation::ui::Application::enable_remove_feature (gui/shared/application.cpp:168)	L174	ambiguous-name
foundation::ui::Application::retry_presentation (gui/shared/application.cpp:296)	L297	ambiguous-name

Print scope: 12 additional caller symbols containing 13 candidate call sites are omitted. Map-linked and same-file callers are listed first. [All callers in explorer](#)

Outgoing: 39 detected call sites grouped into 21 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
lookup L303, L314, L315, L316, L341, L342, L350, L351, L352, L353, L354, L355	foundation::ui::Application::lookup (gui/shared/application.cpp:43) Example::lookup (@gui-boundary/examples/application.hpp:137)	ambiguous-name

Call name / call sites	Candidate target names / captured locations	Analysis
child L330	foundation::ui::Application::publish::child (gui/shared/application.cpp:322)	name-candidate
list.records.clear L304	No indexed target; external / unresolved.	unresolved
entries_.snapshot L305	foundation::Store::snapshot (src/store.cpp:72)	name-candidate
std::to_string L307, L314	No indexed target; external / unresolved.	unresolved
list.records.push_back L310	No indexed target; external / unresolved.	unresolved
std::move L310, L358	No indexed target; external / unresolved.	unresolved
std::none_of L312	No indexed target; external / unresolved.	unresolved

Print scope: 13 additional outgoing groups containing 18 call sites are omitted. Groups containing map-linked or same-file candidates are listed first. [All outgoing calls in explorer](#)

Complete indexed structural outline

54 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	presentation_pending_ = true;	L301
0: statement	auto next = view_;	L302
0: statement	auto& list = lookup(next, "entries.list").state;	L303
0: statement	list.records.clear();	L304
0: loop	for (const auto& entry : entries_.snapshot())	L305
1: statement	gui::Record record;	L306
1: statement	record.id = std::to_string(entry.id);	L307
1: statement	record.accessible_text = entry.text;	L308
1: statement	record.cells = {{entry.text, {}, {}}};	L309
1: statement	list.records.push_back(std::move(record));	L310
0: branch	(list.selected && std::none_of(list.records.begin(), list.records.end(), [&](const gui::Record& row) { return list.selected == row.id; })))	L312
1: then	then	L313
2: statement	list.selected.reset();	L313

Depth / structure	Captured label	Source
0: statement	lookup(next, "entries.count").state.text = std::to_string(list.records.size()) + " entries";	L314
0: statement	lookup(next, "entries.status").state.text = status_;	L315
0: statement	lookup(next, "entries.status").state.font.tone = status_error_ ? gui::Tone::error : gui::Tone::muted;	L316
1: branch	status_error_	L316
2: then	then	L316
3: statement	gui::Tone::error	L316
2: else	else	L316
3: statement	gui::Tone::muted	L316
0: statement	gui::LayoutNode panel;	L317
0: statement	panel.id = "entries.form";	L318
0: statement	panel.kind = gui::LayoutKind::column;	L319
0: statement	panel.padding = {8, 8, 8, 8};	L320
0: statement	panel.gap = 4;	L321
0: statement	const auto child = [&](const std::string& id, double height) { gui::LayoutNode node; node.id = id; node.height = height; panel.children.push_back(std::move(node)); };	L322
0: loop	for (const auto& definition : view_definition)	L328
1: branch	(definition.id != panel.id && (!definition.remove_extension remove_feature_))	L329
2: then	then	L330
3: statement	child(std::string(definition.id), definition.height);	L330
0: statement	const auto width = std::max(0.0, next.client_size.width - 32);	L331
0: statement	const auto height = std::max(0.0, next.client_size.height - 32);	L332
0: statement	const auto boxes = gui::compose_layout(panel, {16, 16, width, height}, [&](std::string_view id, double available) { const auto& widget = lookup(next, id); if (widget.spec.kind !=...	L333
0: loop	for (const auto& box : gui::flatten_layout(boxes))	L340
1: statement	lookup(next, box.id).state.bounds = box.bounds;	L341
0: statement	auto& group = lookup(next, "entries.form").state;	L342
0: statement	group.content_size = {boxes.content.width, boxes.content.height};	L343
0: statement	group.bounds.height = std::min(height, boxes.bounds.height);	L344
0: statement	group.content_clip = gui::Rect{8, 8, std::max(0.0, width - 16), std::max(0.0, group.bounds.height - 16)};	L345

Depth / structure	Captured label	Source
0: statement	list.content_size.width = list.bounds.width;	L347
0: loop	for (auto& record : list.records)	L348
1: statement	record.cells.front().bounds = {4, 0, std::max(0.0, list.bounds.width - 8), 32};	L349
0: statement	lookup(next, "entries.add").state.enabled = !lookup(next, "entries.editor").state.text.empty() && list.records.size() < entry_limit_;	L350
0: branch	(remove_feature_)	L352
1: then	then	L352
2: statement	lookup(next, "entries.remove").state.enabled = list.selected.has_value();	L352
0: statement	lookup(next, "entries.task.start").state.enabled = !task_running_;	L353
0: statement	lookup(next, "entries.task.cancel").state.enabled = task_running_;	L354
0: statement	lookup(next, "entries.task.status").state.text = task_status_;	L355
0: statement	++next.revision;	L356
0: statement	adapter_.present(next);	L357
0: statement	view_ = std::move(next);	L358
0: statement	presentation_pending_ = false;	L359

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::add

method | [gui/shared/application.cpp:29](#) | [Lines 29-41](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui::Application

Captured signature

gui::Widget& Application::add(const ViewDefinition& definition)

Why this implementation is on an edit path

[widget-placement / N1: Where should it appear? \(DECISION\)](#)

Why: The placement choice determines whether a row edit is enough.

[widget-placement / N3: Change grouping or spacing \(EDIT\)](#)

Why: The layout tree computes bounds; WidgetSpec.parent supplies containment and clipping. Both must agree.

[widget-placement / N4: Assign page and parent \(EDIT\)](#)

Why: Names such as entries.custom are widget IDs only; they do not establish tab membership.

[tabs / N3: 3. Create widgets on their page \(EDIT\)](#)

Why: Without this edit, every widget still belongs to entries.form, regardless of the intended tab.

Complete captured source

gui/shared/application.cpp | Lines 29-41 | SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
29  gui::Widget& Application::add(const ViewDefinition& definition) {
30      gui::Widget widget;
31      widget.spec.key.id = definition.id;
32      widget.spec.kind = definition.kind;
33      if (definition.id != view_definition.front().id) widget.spec.parent = view_definition.front().id;
34      widget.state.text = definition.text;
35      widget.state.label = definition.label;
36      widget.state.placeholder = definition.placeholder;
37      widget.state.font.bold = definition.bold;
38      widget.state.wrap = definition.wrap;
39      view_.widgets.push_back(std::move(widget));
40      return view_.widgets.back();
41  }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 47 indexed candidate call sites across 19 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::Application (gui/shared/application.cpp:12)	L17	ambiguous-name
foundation::ui::Application::append_entry (gui/shared/application.cpp:49)	L54	ambiguous-name
foundation::ui::Application::complete_service (gui/shared/application.cpp:232)	L253	ambiguous-name
Example::Example (@gui-boundary/examples/application.hpp:12)	L15, L16, L17, L20, L23, L24, L25, L28, L31, L35, L36, L37, L38, L39, L42	ambiguous-name
anonymous namespace @12::run (src/main.cpp:13)	L21, L36	ambiguous-name
foundation::ui::Application::enable_remove_feature (gui/shared/application.cpp:168)	L172	ambiguous-name
foundation::host::Bezel::capture (gui/host/bezel.hpp:124)	L155, L156, L158, L160	ambiguous-name
main (examples/consumer/main.cpp:12)	L17	ambiguous-name

Print scope: 11 additional caller symbols containing 21 candidate call sites are omitted. Map-linked and same-file callers are listed first. [All callers in explorer](#)

Outgoing: 5 detected call sites grouped into 4 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
<code>view_definition.front</code> L33	No indexed target; external / unresolved.	unresolved
<code>view_.widgets.push_back</code> L39	No indexed target; external / unresolved.	unresolved
<code>std::move</code> L39	No indexed target; external / unresolved.	unresolved
<code>view_.widgets.back</code> L40	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

13 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	<code>gui::Widget widget;</code>	L30
0: statement	<code>widget.spec.key.id = definition.id;</code>	L31
0: statement	<code>widget.spec.kind = definition.kind;</code>	L32
0: branch	<code>(definition.id != view_definition.front().id)</code>	L33
1: then	<code>then</code>	L33
2: statement	<code>widget.spec.parent = view_definition.front().id;</code>	L33
0: statement	<code>widget.state.text = definition.text;</code>	L34
0: statement	<code>widget.state.label = definition.label;</code>	L35
0: statement	<code>widget.state.placeholder = definition.placeholder;</code>	L36
0: statement	<code>widget.state.font.bold = definition.bold;</code>	L37
0: statement	<code>widget.state.wrap = definition.wrap;</code>	L38
0: statement	<code>view_.widgets.push_back(std::move(widget));</code>	L39
0: return	<code>return view_.widgets.back();</code>	L40

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::Rect

struct | @gui-boundary/include/gui/geometry.hpp:13 | Lines 13-16 | Interactive symbol / full call lists

Owner: gui

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: third_party/gui-inputs/gui-inputs.tar.gz | member: upstream/include/gui/geometry.hpp | revision: 7a704f73e563a167ea335dd23ccd9f383ebec274

Captured signature

struct Rect

Why this implementation is on an edit path

widget-placement / N3: Change grouping or spacing (EDIT)

Why: The layout tree computes bounds; WidgetSpec.parent supplies containment and clipping. Both must agree.

widget-placement / N5: Let shared layout place it (REUSE)

Why: Changing a backend-specific pixel position would bypass the shared placement source.

tabs / N4: 4. Lay out content and tab bar (EDIT)

Why: Declaring Page entries alone does not allocate visible tab-bar space or place their contents.

Owning type: 1 indexed members in this declaration file. Members not recognized by the syntax scanner remain visible in the complete source below.

Indexed member	Kind / line	Signature
gui::Rect::operator==	method L15	bool operator==(const Rect&) const = default;

Complete captured source

@gui-boundary/include/gui/geometry.hpp | Lines 13-16 | SHA-256 e5b8e7adcda1c4e268bbb374b22bdf13706c97cca0e414d78ad747da2336958e

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
13 | struct Rect {
14 |     double x=0, y=0, width=0, height=0;
15 |     bool operator==(const Rect&) const = default;
16 | };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

1 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	double x=0, y=0, width=0, height=0;	L14

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

Example::publish

[method](#) | [@gui-boundary/examples/application.hpp:150](#) | [Lines 150-188](#) | [Interactive symbol / full call lists](#)

Owner: Example

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/examples/application.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 132 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

void publish()

Why this implementation is on an edit path

[widget-placement / N3: Change grouping or spacing \(EDIT\)](#)

Why: The layout tree computes bounds; WidgetSpec.parent supplies containment and clipping. Both must agree.

[widget-placement / N5: Let shared layout place it \(REUSE\)](#)

Why: Changing a backend-specific pixel position would bypass the shared placement source.

[tabs / N4: 4. Lay out content and tab bar \(EDIT\)](#)

Why: Declaring Page entries alone does not allocate visible tab-bar space or place their contents.

Complete captured source

[@gui-boundary/examples/application.hpp](#) | [Lines 150-188](#) | [SHA-256 45260f74dcf1d5c6c24514130d58c197d70711d6e06bab283832ce1fd0b3cbf4](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
150 void publish() {
151     // Values and accepted service replies remain authoritative on failure.
152     // Stage layout separately and retain repaint debt for the host's next
153     // tick/error recovery. A smaller resize can replace a failed large one.
154     presentation_pending=true;
155     auto next=view_;
156     next.page_bar={16,std::max(0.0,next.client_size.height-32),std::max(0.0,next.client_size.width-32),24};
157     // Modal placement is shared and remains reachable after a small resize.
```

```
158     const double mx=std::min(16.0,next.client_size.width/8),my=std::min(16.0,next.client_size.height/8);
159     const double dw=std::min(416.0,std::max(0.0,next.client_size.width-2*mx));
160     const double dh=std::min(240.0,std::max(0.0,next.client_size.height-2*my));
161     const double dx=(next.client_size.width-dw)/2,dy=(next.client_size.height-dh)/2;
162     lookup(next,"dialog").state.bounds={dx,dy,dw,dh};
163     lookup(next,"dialog-title").state.bounds={dx+std::min(24.0,dw/8),dy+std::min(16.0,dh/8),dw*.85,std::min(32.0,dh/4)}
164 + ;
165     lookup(next,"dialog-body").state.bounds={dx+dw*.06,dy+dh*.25,dw*.88,dh*.35};
166     lookup(next,"dismiss").state.bounds={dx+dw*.54,dy+dh*.73,dw*.4,dh*.2};
167     gui::LayoutNode panel;panel.id="panel";panel.kind=gui::LayoutKind::column;
168     panel.padding={8,8,8,8};panel.gap=8;
169     for(const auto* id:{"heading","choice","editor","toggle","button","menu","list"}) {
170         gui::LayoutNode child;child.id=id;
171         if(child.id!="heading")child.height=child.id=="list"?112:28;
172         panel.children.push_back(std::move(child));
173     }
174     const auto width=std::min(384.0,std::max(0.0,next.client_size.width-32));
175     const auto height=std::max(0.0,next.client_size.height-64);
176     const auto layout=gui::compose_layout(panel,{16,16,width,height},{&}(std::string_view id,double available) {
177         const auto& w=lookup(next,id);
178         if(w.spec.kind!=gui::Kind::label)return gui::Size{available,0};
179         return adapter_.measure_text({w.state.text,w.state.font,available,next.display_scale,w.state.wrap});
180     });
181     for(const auto& box:gui::flatten_layout(layout))lookup(next,box.id).state.bounds=box.bounds;
182     auto& group=lookup(next,"panel").state;
183     group.content_size={layout.content.width,layout.content.height};
184     group.bounds.height=std::min(height,layout.bounds.height);
185     group.content_clip=gui::Rect{8,8,std::max(0.0,group.bounds.width-16),std::max(0.0,group.bounds.height-16)};
186     lookup(next,"button").state.enabled=lookup(next,"toggle").state.checked;
187     ++next.revision;adapter_.present(next);
188     view_=std::move(next);presentation_pending_=false;
189 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 23 indexed candidate call sites across 20 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
Example::Example (@gui-boundary/examples/application.hpp:12)	L44	ambiguous-name
Example::handle (@gui-boundary/examples/application.hpp:67)	L113, L119, L121	ambiguous-name
foundation::ui::Application::Application (gui/shared/application.cpp:12)	L26	ambiguous-name

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::handle (gui/shared/application.cpp:74)	L165	ambiguous-name
foundation::ui::Application::complete_service (gui/shared/application.cpp:232)	L262	ambiguous-name
foundation::ui::Application::complete_task (gui/shared/application.cpp:281)	L292	ambiguous-name
Example::complete_service (@gui-boundary/examples/application.hpp:48)	L51	ambiguous-name
Example::retry_presentation (@gui-boundary/examples/application.hpp:61)	L61	ambiguous-name

Print scope: 12 additional caller symbols containing 13 candidate call sites are omitted. Map-linked and same-file callers are listed first. [All callers in explorer](#)

Outgoing: 31 detected call sites grouped into 8 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
lookup L162, L163, L164, L165, L180, L181, L185	Example::lookup (@gui-boundary/examples/application.hpp:137) foundation::ui::Application::lookup (gui/shared/application.cpp:43)	ambiguous-name
std::max L156, L159, L160, L173, L174, L184	No indexed target; external / unresolved.	unresolved
std::min L158, L159, L160, L163, L173, L183	No indexed target; external / unresolved.	unresolved
panel.children.push_back L171	No indexed target; external / unresolved.	unresolved
std::move L171, L187	No indexed target; external / unresolved.	unresolved
gui::compose_layout L175	No indexed target; external / unresolved.	unresolved
gui::flatten_layout L180	No indexed target; external / unresolved.	unresolved
adapter_.present L186	foundation::host::FramebufferHost::present (gui/host/framebuffer.hpp:97) anonymous namespace @18::present (tools/windows_gl_probe.cpp:103) anonymous namespace @9::FailingAdapter::present (gui/tests/application_test.cpp:160) +4 additional candidate targets omitted; all targets	ambiguous-name

Complete indexed structural outline

42 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	presentation_pending_=true;	L154
0: statement	auto next=view_;	L155
0: statement	next.page_bar={16,std::max(0.0,next.client_size.height-32),std::max(0.0,next.client_size.width-32),24};	L156

Depth / structure	Captured label	Source
0: statement	const double mx=std::min(16.0,next.client_size.width/8),my=std::min(16.0,next.client_size.height/8);	L158
0: statement	const double dw=std::min(416.0,std::max(0.0,next.client_size.width-2*mx));	L159
0: statement	const double dh=std::min(240.0,std::max(0.0,next.client_size.height-2*my));	L160
0: statement	const double dx=(next.client_size.width-dw)/2,dy=(next.client_size.height-dh)/2;	L161
0: statement	lookup(next,"dialog").state.bounds={dx,dy,dw,dh};	L162
0: statement	lookup(next,"dialog-title").state.bounds={dx+std::min(24.0,dw/8),dy+std::min(16.0,dh/8),dw*.85,std::min(32.0,dh/4)};	L163
0: statement	lookup(next,"dialog-body").state.bounds={dx+dw*.06,dy+dh*.25,dw*.88,dh*.35};	L164
0: statement	lookup(next,"dismiss").state.bounds={dx+dw*.54,dy+dh*.73,dw*.4,dh*.2};	L165
0: statement	gui::LayoutNode panel;	L166
0: statement	panel.id="panel";	L166
0: statement	panel.kind=gui::LayoutKind::column;	L166
0: statement	panel.padding={8,8,8,8};	L167
0: statement	panel.gap=8;	L167
0: loop	for(const auto* id>{"heading","choice","editor","toggle","button","menu","list"})	L168
1: statement	gui::LayoutNode child;	L169
1: statement	child.id=id;	L169
1: branch	(child.id!="heading")	L170
2: then	then	L170
3: statement	child.height=child.id=="list"?112:28;	L170
4: branch	child.id=="list"	L170
5: then	then	L170
6: statement	112	L170
5: else	else	L170
6: statement	28	L170
1: statement	panel.children.push_back(std::move(child));	L171
0: statement	const auto width=std::min(384.0,std::max(0.0,next.client_size.width-32));	L173
0: statement	const auto height=std::max(0.0,next.client_size.height-64);	L174
0: statement	const auto layout=gui::compose_layout(panel,{16,16,width,height},[&](std::string_view id,double available) { const auto& w=lookup(next,id); if(w.spec.kind!=gui::Kind::label)return...	L175

Depth / structure	Captured label	Source
0: loop	for(const auto& box:gui::flatten_layout(layout))	L180
1: statement	lookup(next,box.id).state.bounds=box.bounds;	L180
0: statement	auto& group=lookup(next,"panel").state;	L181
0: statement	group.content_size={layout.content.width,layout.content.height};	L182
0: statement	group.bounds.height=std::min(height,layout.bounds.height);	L183
0: statement	group.content_clip=gui::Rect{8,8,std::max(0.0,group.bounds.width-16),std::max(0.0,group.bounds.height-16)};	L184
0: statement	lookup(next,"button").state.enabled=lookup(next,"toggle").state.checked;	L185
0: statement	++next.revision;	L186
0: statement	adapter_.present(next);	L186
0: statement	view_=std::move(next);	L187
0: statement	presentation_pending_=false;	L187

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::WidgetState

[struct](#) | [@gui-boundary/include/gui/contract.hpp:93](#) | [Lines 93-111](#) | [Interactive symbol](#) / [full call lists](#)

Owner: [gui](#)

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/contract.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing type_identifier), line 330 (missing type_identifier), line 371 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

```
struct WidgetState
```

Why this implementation is on an edit path

[widget-placement / N3: Change grouping or spacing \(EDIT\)](#)

Why: The layout tree computes bounds; WidgetSpec.parent supplies containment and clipping. Both must agree.

[widget-placement / N5: Let shared layout place it \(REUSE\)](#)

Why: Changing a backend-specific pixel position would bypass the shared placement source.

[tabs / N4: 4. Lay out content and tab bar \(EDIT\)](#)

Why: Declaring Page entries alone does not allocate visible tab-bar space or place their contents.

Complete captured source

@gui-boundary/include/gui/contract.hpp | Lines 93-111 | SHA-256 f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00ffd9db629a404d40da8

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
93 struct WidgetState {
94     Rect bounds;
95     // A group's optional child viewport, relative to its own top-left corner.
96     // Its frame remains available; only descendants use this additional clip.
97     std::optional<Rect> content_clip;
98     std::string label,text,help,accessible_name,placeholder,display_text;
99     Font font;
100     TextWrap wrap=TextWrap::none;
101     bool enabled=true,visible=true,checked=false;
102     std::optional<std::string> selected;
103     std::vector<Option> options;
104     // Named bitmap actions exposed through native keyboard/accessibility UI.
105     std::vector<Option> actions;
106     std::vector<Record> records;
107     // Scrollable content extent measured from the child viewport's top-left.
108     // A group's content_clip determines viewport size; padding is not content.
109     Size content_size;
110     BitmapView bitmap;
111 };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

12 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	Rect bounds;	L94
0: statement	std::optional<Rect> content_clip;	L97
0: statement	std::string label,text,help,accessible_name,placeholder,display_text;	L98

Depth / structure	Captured label	Source
0: statement	Font font;	L99
0: statement	TextWrap wrap=TextWrap::none;	L100
0: statement	bool enabled=true,visible=true,checked=false;	L101
0: statement	std::optional<std::string> selected;	L102
0: statement	std::vector<Option> options;	L103
0: statement	std::vector<Option> actions;	L105
0: statement	std::vector<Record> records;	L106
0: statement	Size content_size;	L109
0: statement	BitmapView bitmap;	L110

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::WidgetSpec

`struct` | [@gui-boundary/include/gui/contract.hpp:79](#) | [Lines 79-87](#) | [Interactive symbol](#) / [full call lists](#)

Owner: `gui`

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/contract.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing `type_identifier`), line 330 (missing `type_identifier`), line 371 (missing `type_identifier`). Recovered structure may be incomplete.

Captured signature

```
struct WidgetSpec
```

Why this implementation is on an edit path

[widget-placement / N4: Assign page and parent \(EDIT\)](#)

Why: Names such as `entries.custom` are widget IDs only; they do not establish tab membership.

[tabs / N2: 2. Declare page-owned content \(EDIT\)](#)

Why: The current row schema has no page or parent field; an ID prefix cannot select a tab.

[tabs / N3: 3. Create widgets on their page \(EDIT\)](#)

Why: Without this edit, every widget still belongs to `entries.form`, regardless of the intended tab.

Owning type: 1 indexed members in this declaration file. Members not recognized by the syntax scanner remain visible in the complete source below.

Indexed member	Kind / line	Signature
gui::WidgetSpec::operator==	method L86	bool operator==(const WidgetSpec&) const = default;

Complete captured source

[@gui-boundary/include/gui/contract.hpp](#) | Lines 79-87 | SHA-256 f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00fd9db629a404d40da8

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
79 struct WidgetSpec {
80     WidgetKey key;
81     Kind kind=Kind::label;
82     std::string page,parent,binding;
83     TextPolicy text_policy;
84     bool activate_on_select=false,follow_tail=false,pointer_input=false;
85     double row_height=28;
86     bool operator==(const WidgetSpec&) const = default;
87 };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

6 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	WidgetKey key;	L80
0: statement	Kind kind=Kind::label;	L81
0: statement	std::string page,parent,binding;	L82
0: statement	TextPolicy text_policy;	L83
0: statement	bool activate_on_select=false,follow_tail=false,pointer_input=false;	L84
0: statement	double row_height=28;	L85

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::validate_snapshot

[function](#) | [@gui-boundary/include/gui/contract.hpp:234](#) | Lines 234-324 | [Interactive symbol / full call lists](#)

Owner: gui

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/contract.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing type_identifier), line 330 (missing type_identifier), line 371 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

```
inline void validate_snapshot(const Snapshot& view)
```

Why this implementation is on an edit path

[widget-placement / N4: Assign page and parent \(EDIT\)](#)

Why: Names such as `entries.custom` are widget IDs only; they do not establish tab membership.

[tabs / N2: 2. Declare page-owned content \(EDIT\)](#)

Why: The current row schema has no page or parent field; an ID prefix cannot select a tab.

[tabs / N3: 3. Create widgets on their page \(EDIT\)](#)

Why: Without this edit, every widget still belongs to `entries.form`, regardless of the intended tab.

[tabs / N7: 7. Reuse adapter focus handling \(REUSE\)](#)

Why: The existing adapter owns focus and availability. Add application focus policy only when the feature needs it.

[commands / N4: 4. Route a current button \(EDIT\)](#)

Why: The retained contract requires every shortcut to target a current button, not a menu option ID.

[commands / N5: 5. Declare a supported key binding \(EDIT\)](#)

Why: A shortcut is declared in the shared Snapshot, independent of a native toolkit callback.

Complete captured source

[@gui-boundary/include/gui/contract.hpp](#) | Lines 234-324 | SHA-256 [f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00ffd9db629a404d40da8](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
234 inline void validate_snapshot(const Snapshot& view) {
235     const auto require=[](bool ok,const char* message){if(!ok)throw std::invalid_argument(message);};
236     const auto text=[&](const std::string& s){require(valid_utf8(s),"Invalid UTF-8 presentation text");};
237     require(valid_rect({0,0,view.client_size.width,view.client_size.height}),"Invalid client size");
238     require(valid_rect(view.page_bar),"Invalid page navigation bounds");
239     device_rect({0,0,view.client_size.width,view.client_size.height},view.display_scale);
240     (void)pixel_row_bytes(0,view.bitmap_format);
241     text(view.title);
242     std::set<std::string> pages;
```

```

243     for(const auto& page:view.pages) {
244         text(page.id);text(page.label);
245         require(!page.id.empty())&&pages.insert(page.id).second,"Page IDs must be nonempty and unique");
246     }
247     require(!view.active_page||pages.contains(*view.active_page),"Unknown active page");
248     std::map<std::string,const Widget*,std::less<>> widgets;
249     for(const auto& w:view.widgets) {
250         const auto& s=w.spec;const auto& v=w.state;
251         switch(s.kind) {
252             case Kind::group:case Kind::label:case Kind::button:case Kind::toggle:case Kind::choice:
253             case Kind::text:case Kind::list:case Kind::bitmap:case Kind::menu;break;
254             default:throw std::invalid_argument("Unknown widget kind");
255         }
256         switch(s.text_policy.submit) {
257             case SubmitKey::none:case SubmitKey::enter:case SubmitKey::control_enter;break;
258             default:throw std::invalid_argument("Unknown text submission policy");
259         }
260         text(s.key.id);text(s.page);text(s.parent);text(s.binding);
261         require(!s.key.id.empty())&&s.key.generation!=0&&widgets.emplace(s.key.id,&w).second,"Invalid or duplicate widget
+ key");
262         require(s.page.empty()||pages.contains(s.page),"Unknown widget page");
263         if(!s.parent.empty()) {
264             const auto found=widgets.find(s.parent);
265             require(found!=widgets.end())&&found->second!=&w&&found->second->spec.kind==Kind::group,"Parent must be an
+ earlier group");
266             require(found->second->spec.page==s.page,"Child and parent must use the same page");
267         }
268         require(valid_rect(v.bounds)&&valid_rect({0,0,v.content_size.width,v.content_size.height}),"Invalid widget
+ geometry");
269         validate_font(v.font);validate_wrap(v.wrap);
270         if(v.content_clip)require(s.kind==Kind::group&&valid_rect(*v.content_clip),"Invalid group content clip");
271         for(const auto* value:{&v.label,&v.text,&v.help,&v.accessible_name,&v.placeholder,&v.display_text})text(*value);
272         require(std::isfinite(s.row_height)&&s.row_height>0&&s.row_height<=coordinate_limit,"Invalid row height");
273         if(s.kind==Kind::text)require(text_error(v.text,s.text_policy).empty(),"Invalid editor state");
274         require(v.options.empty()||s.kind==Kind::choice||s.kind==Kind::menu||s.kind==Kind::text,"Options on unsupported
+ widget");
275         require(v.actions.empty()||s.kind==Kind::bitmap,"Actions on unsupported widget");
276         require(v.records.empty()||s.kind==Kind::list,"Records on unsupported widget");
277         require(!v.selected||s.kind==Kind::list||s.kind==Kind::choice,"Selection on unsupported widget");
278         if(v.selected){text(*v.selected);require(!v.selected->empty(),"Empty selected ID");}
279         std::set<std::string> ids;
280         for(const auto& option:v.options) {
281             text(option.id);text(option.label);text(option.value);
282             require(!option.id.empty())&&ids.insert(option.id).second,"Option IDs must be nonempty and unique");
283             if(s.kind==Kind::text)require(text_error(option.value,s.text_policy).empty(),"Invalid suggested text");

```

```

284     }
285     ids.clear();
286     for(const auto& action:v.actions) {
287         text(action.id);text(action.label);
288         require(!action.id.empty()&&!action.label.empty()&&action.value.empty()&&ids.insert(action.id).second,
289             "Action IDs and labels must be nonempty, IDs unique, and values empty");
290     }
291     ids.clear();
292     require(double(v.records.size()*s.row_height<=coordinate_limit,"List is too tall");
293     for(const auto& record:v.records) {
294         text(record.id);text(record.accessible_text);
295         require(!record.id.empty()&&ids.insert(record.id).second,"Record IDs must be nonempty and unique");
296         for(const auto& cell:record.cells) {
297             text(cell.text);validate_font(cell.font);validate_wrap(cell.wrap);require(valid_rect(cell.bounds),"Invalid
+ record cell geometry");
298         }
299     }
300     text(v.bitmap.source_id);
301 }
302 if(view.modal_root) {
303     const auto* root=find_widget(view,*view.modal_root);
304     require(root&&root->spec.kind==Kind::group,"Modal root must name a current group");
305     for(auto* current=root;current;) {
306         require(current->state.visible&&current->state.enabled,"Modal root and ancestors must be visible and enabled");
307         if(!current->spec.page.empty()) {
308             const auto page=std::find_if(view.pages.begin(),view.pages.end(),[&](const Page& p){return
+ p.id==current->spec.page;});
309             require(page!=view.pages.end()&&page->visible&&page->enabled&&view.active_page==page->id,
310                 "Modal root must be on an available active page");
311         }
312         if(current->spec.parent.empty())break;
313         const auto parent=std::find_if(view.widgets.begin(),view.widgets.end(),[&](const Widget& w){return
+ w.spec.key.id==current->spec.parent;});
314         current=parent==view.widgets.end()?nullptr:&*parent;
315     }
316 }
317 std::set<std::tuple<ShortcutKey,bool,bool,bool>> strokes;
318 for(const auto& binding:view.key_bindings) {
319     require(binding.key>=ShortcutKey::escape&&binding.key<=ShortcutKey::f12,"Unknown shortcut key");
320     const auto* target=find_widget(view,binding.target);
321     require(target&&target->spec.kind==Kind::button,"Shortcut must target a current button");
322     require(strokes.emplace(binding.key,binding.control,binding.shift,binding.alt).second,"Duplicate shortcut");
323 }
324 }

```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 105 detected call sites grouped into 43 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
require L237, L238, L245, L247, L261, L262, L265, L266, L268, L270, L272, L273, L274, L275, L276, L277, L278, L282, L283, L288, L292, L295, L297, L304, L306, L309, L319, L321, L322	gui::validate_snapshot::require (@gui-boundary/include/gui/contract.hpp:235) foundation::ui::Application::qualify::require (gui/shared/application.cpp:178) anonymous namespace @9::require (tests/rust_component_status_test.cpp:12) +2 additional candidate targets omitted ; all targets	ambiguous-name
text L241, L244, L260, L271, L278, L281, L287, L294, L297, L300	gui::validate_snapshot::text (@gui-boundary/include/gui/contract.hpp:236) anonymous namespace @4::text (gui/tests/sdl_test.cpp:9)	ambiguous-name
validate_font L269, L297	gui::validate_font (@gui-boundary/include/gui/contract.hpp:43)	name-candidate
validate_wrap L269, L297	gui::validate_wrap (@gui-boundary/include/gui/contract.hpp:51)	name-candidate
find_widget L303, L320	gui::find_widget (@gui-boundary/include/gui/contract.hpp:166)	name-candidate
valid_rect L237, L238, L268, L270, L297	gui::valid_rect (@gui-boundary/include/gui/geometry.hpp:18)	name-candidate
device_rect L239	gui::device_rect (@gui-boundary/include/gui/geometry.hpp:44)	name-candidate
pixel_row_bytes L240	No indexed target; external / unresolved.	unresolved

Print scope: 35 additional outgoing groups containing 44 call sites are omitted. Groups containing map-linked or same-file candidates are listed first. [All outgoing calls in explorer](#)

Complete indexed structural outline

120 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	const auto require=[](bool ok,const char* message){if(!ok)throw std::invalid_argument(message);};	L235
0: statement	const auto text=[&](const std::string& s){require(valid_utf8(s),"Invalid UTF-8 presentation text");};	L236
0: statement	require(valid_rect({0,0,view.client_size.width,view.client_size.height}),"Invalid client size");	L237

Depth / structure	Captured label	Source
0: statement	<code>require(valid_rect(view.page_bar),"Invalid page navigation bounds");</code>	L238
0: statement	<code>device_rect({0,0,view.client_size.width,view.client_size.height},view.display_scale);</code>	L239
0: statement	<code>(void)pixel_row_bytes(0,view.bitmap_format);</code>	L240
0: statement	<code>text(view.title);</code>	L241
0: statement	<code>std::set<std::string> pages;</code>	L242
0: loop	<code>for(const auto& page:view.pages)</code>	L243
1: statement	<code>text(page.id);</code>	L244
1: statement	<code>text(page.label);</code>	L244
1: statement	<code>require(!page.id.empty()&&pages.insert(page.id).second,"Page IDs must be nonempty and unique");</code>	L245
0: statement	<code>require(!view.active_page pages.contains(*view.active_page),"Unknown active page");</code>	L247
0: statement	<code>std::map<std::string,const Widget*,std::less<>> widgets;</code>	L248
0: loop	<code>for(const auto& w:view.widgets)</code>	L249
1: statement	<code>const auto& s=w.spec;</code>	L250
1: statement	<code>const auto& v=w.state;</code>	L250
1: switch	<code>(s.kind)</code>	L251
2: case	<code>Kind::group</code>	L252
2: case	<code>Kind::label</code>	L252
2: case	<code>Kind::button</code>	L252
2: case	<code>Kind::toggle</code>	L252
2: case	<code>Kind::choice</code>	L252
2: case	<code>Kind::text</code>	L253
2: case	<code>Kind::list</code>	L253
2: case	<code>Kind::bitmap</code>	L253
2: case	<code>Kind::menu</code>	L253
3: break	<code>break;</code>	L253
2: case	<code>default</code>	L254
3: throw	<code>throw std::invalid_argument("Unknown widget kind");</code>	L254
1: switch	<code>(s.text_policy.submit)</code>	L256

Depth / structure	Captured label	Source
2: case	SubmitKey::none	L257
2: case	SubmitKey::enter	L257
2: case	SubmitKey::control_enter	L257
3: break	break;	L257
2: case	default	L258
3: throw	throw std::invalid_argument("Unknown text submission policy");	L258
1: statement	text(s.key.id);	L260
1: statement	text(s.page);	L260
1: statement	text(s.parent);	L260
1: statement	text(s.binding);	L260
1: statement	require(!s.key.id.empty()&& s.key.generation!=0&& widgets.emplace(s.key.id,&w).second,"Invalid or duplicate widget key");	L261
1: statement	require(s.page.empty() pages.contains(s.page),"Unknown widget page");	L262
1: branch	(!s.parent.empty())	L263
2: then	then	L263
3: statement	const auto found=widgets.find(s.parent);	L264
3: statement	require(found!=widgets.end()&& found->second!=&w&& found->second->spec.kind==Kind::group,"Parent must be an earlier group");	L265
3: statement	require(found->second->spec.page==s.page,"Child and parent must use the same page");	L266
1: statement	require(valid_rect(v.bounds)&& valid_rect({0,0,v.content_size.width,v.content_size.height}),"Invalid widget geometry");	L268
1: statement	validate_font(v.font);	L269
1: statement	validate_wrap(v.wrap);	L269
1: branch	(v.content_clip)	L270
2: then	then	L270
3: statement	require(s.kind==Kind::group&& valid_rect(*v.content_clip),"Invalid group content clip");	L270
1: loop	for(const auto* value:{&v.label,&v.text,&v.help,&v.accessible_name,&v.placeholder,&v.display_text})	L271
2: statement	text(*value);	L271
1: statement	require(std::isfinite(s.row_height)&& s.row_height>0&& s.row_height<=coordinate_limit,"Invalid row height");	L272
1: branch	(s.kind==Kind::text)	L273
2: then	then	L273

Depth / structure	Captured label	Source
3: statement	<code>require(text_error(v.text,s.text_policy).empty(),"Invalid editor state");</code>	L273
1: statement	<code>require(v.options.empty() s.kind==Kind::choice s.kind==Kind::menu s.kind==Kind::text,"Options on unsupported widget");</code>	L274
1: statement	<code>require(v.actions.empty() s.kind==Kind::bitmap,"Actions on unsupported widget");</code>	L275
1: statement	<code>require(v.records.empty() s.kind==Kind::list,"Records on unsupported widget");</code>	L276
1: statement	<code>require(!v.selected s.kind==Kind::list s.kind==Kind::choice,"Selection on unsupported widget");</code>	L277
1: branch	<code>(v.selected)</code>	L278
2: then	<code>then</code>	L278
3: statement	<code>text(*v.selected);</code>	L278
3: statement	<code>require(!v.selected->empty(),"Empty selected ID");</code>	L278
1: statement	<code>std::set<std::string> ids;</code>	L279
1: loop	<code>for(const auto& option:v.options)</code>	L280
2: statement	<code>text(option.id);</code>	L281
2: statement	<code>text(option.label);</code>	L281
2: statement	<code>text(option.value);</code>	L281
2: statement	<code>require(!option.id.empty()&&ids.insert(option.id).second,"Option IDs must be nonempty and unique");</code>	L282
2: branch	<code>(s.kind==Kind::text)</code>	L283
3: then	<code>then</code>	L283
4: statement	<code>require(text_error(option.value,s.text_policy).empty(),"Invalid suggested text");</code>	L283
1: statement	<code>ids.clear();</code>	L285
1: loop	<code>for(const auto& action:v.actions)</code>	L286
2: statement	<code>text(action.id);</code>	L287
2: statement	<code>text(action.label);</code>	L287
2: statement	<code>require(!action.id.empty()&&!action.label.empty()&&action.value.empty()&&ids.insert(action.id).second, "Action IDs and labels must be nonempty, IDs unique, and values empty");</code>	L288
1: statement	<code>ids.clear();</code>	L291
1: statement	<code>require(double(v.records.size())*s.row_height<=coordinate_limit,"List is too tall");</code>	L292
1: loop	<code>for(const auto& record:v.records)</code>	L293
2: statement	<code>text(record.id);</code>	L294
2: statement	<code>text(record.accessible_text);</code>	L294

Depth / structure	Captured label	Source
2: statement	<code>require(!record.id.empty())&&ids.insert(record.id).second,"Record IDs must be nonempty and unique");</code>	L295
2: loop	<code>for(const auto& cell:record.cells)</code>	L296
3: statement	<code>text(cell.text);</code>	L297
3: statement	<code>validate_font(cell.font);</code>	L297
3: statement	<code>validate_wrap(cell.wrap);</code>	L297
3: statement	<code>require(valid_rect(cell.bounds),"Invalid record cell geometry");</code>	L297
1: statement	<code>text(v.bitmap.source_id);</code>	L300
0: branch	<code>(view.modal_root)</code>	L302
1: then	<code>then</code>	L302
2: statement	<code>const auto* root=find_widget(view,*view.modal_root);</code>	L303
2: statement	<code>require(root&&root->spec.kind==Kind::group,"Modal root must name a current group");</code>	L304
2: loop	<code>for(auto* current=root;current;)</code>	L305
3: statement	<code>require(current->state.visible&&current->state.enabled,"Modal root and ancestors must be visible and enabled");</code>	L306
3: branch	<code>(!current->spec.page.empty())</code>	L307
4: then	<code>then</code>	L307
5: statement	<code>const auto page=std::find_if(view.pages.begin(),view.pages.end(),[&](const Page& p){return p.id==current->spec.page;});</code>	L308
5: statement	<code>require(page!=view.pages.end()&&page->visible&&page->enabled&&view.active_page==page->id, "Modal root must be on an available active page");</code>	L309
3: branch	<code>(current->spec.parent.empty())</code>	L312
4: then	<code>then</code>	L312
5: break	<code>break;</code>	L312
3: statement	<code>const auto parent=std::find_if(view.widgets.begin(),view.widgets.end(),[&](const Widget& w){return w.spec.key.id==current->spec.parent;});</code>	L313
3: statement	<code>current=parent==view.widgets.end()?nullptr:&*parent;</code>	L314
4: branch	<code>parent==view.widgets.end()</code>	L314
5: then	<code>then</code>	L314
6: statement	<code>nullptr</code>	L314
5: else	<code>else</code>	L314
6: statement	<code>&*parent</code>	L314

Depth / structure	Captured label	Source
0: statement	std::set<std::tuple<ShortcutKey,bool,bool,bool>> strokes;	L317
0: loop	for(const auto& binding:view.key_bindings)	L318
1: statement	require(binding.key>=ShortcutKey::escape&&binding.key<=ShortcutKey::f12,"Unknown shortcut key");	L319
1: statement	const auto* target=find_widget(view,binding.target);	L320
1: statement	require(target&&target->spec.kind==Kind::button,"Shortcut must target a current button");	L321
1: statement	require(strokes.emplace(binding.key,binding.control,binding.shift,binding.alt).second,"Duplicate shortcut");	L322

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::WidgetKey

struct | [@gui-boundary/include/gui/contract.hpp:19](#) | [Lines 19-23](#) | [Interactive symbol](#) / [full call lists](#)

Owner: `gui`

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: `third_party/gui-inputs/gui-inputs.tar.gz` | member: `upstream/include/gui/contract.hpp` | revision: `7a704f73e563a167ea335dd23ccd9f383ebec274`

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing type_identifier), line 330 (missing type_identifier), line 371 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

```
struct WidgetKey
```

Why this implementation is on an edit path

[widget-placement / N4: Assign page and parent \(EDIT\)](#)

Why: Names such as `entries.custom` are widget IDs only; they do not establish tab membership.

[tabs / N2: 2. Declare page-owned content \(EDIT\)](#)

Why: The current row schema has no page or parent field; an ID prefix cannot select a tab.

[tabs / N3: 3. Create widgets on their page \(EDIT\)](#)

Why: Without this edit, every widget still belongs to `entries.form`, regardless of the intended tab.

[commands / N5: 5. Declare a supported key binding \(EDIT\)](#)

Why: A shortcut is declared in the shared Snapshot, independent of a native toolkit callback.

Owning type: 1 indexed members in this declaration file. Members not recognized by the syntax scanner remain visible in the complete source below.

Indexed member	Kind / line	Signature
<code>gui::WidgetKey::operator==</code>	method L22	<code>bool operator==(const WidgetKey&) const = default;</code>

Complete captured source

@gui-boundary/include/gui/contract.hpp | Lines 19-23 | SHA-256 f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00ffd9db629a404d40da8

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
19 struct WidgetKey {
20     std::string id;
21     std::uint64_t generation=1;
22     bool operator==(const WidgetKey&) const = default;
23 };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

2 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	std::string id;	L20
0: statement	std::uint64_t generation=1;	L21

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::PageEvent

struct | @gui-boundary/include/gui/contract.hpp:159 | Lines 159-159 | [Interactive symbol](#) / [full call lists](#)

Owner: gui

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/contract.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing type_identifier), line 330 (missing type_identifier), line 371 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

struct PageEvent

Why this implementation is on an edit path

[widget-placement / N7: No tabs yet? Add the page model \(JUMP\)](#)
Why: Another ViewDefinition row alone cannot create a tab in this application.

[tabs / N5: 5. Handle a page switch \(EDIT\)](#)
Why: The current handler has no PageEvent branch. Normalization already rejects unavailable pages, the active page, and page switches while a modal root is active.

Complete captured source

[@gui-boundary/include/gui/contract.hpp](#) | Lines 159-159 | SHA-256 f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00ffd9db629a404d40da8
Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
159 | struct PageEvent {std::string id};;
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

1 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	std::string id;	L159

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::Snapshot

[struct](#) | [@gui-boundary/include/gui/contract.hpp:124](#) | [Lines 124-141](#) | [Interactive symbol](#) / [full call lists](#)

Owner: `gui`

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/contract.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing `type_identifier`), line 330 (missing `type_identifier`), line 371 (missing `type_identifier`). Recovered structure may be incomplete.

Captured signature

```
struct Snapshot
```

Why this implementation is on an edit path

[widget-placement](#) / N7: No tabs yet? Add the page model (JUMP)

Why: Another `ViewDefinition` row alone cannot create a tab in this application.

[tabs](#) / N4: 4. Lay out content and tab bar (EDIT)

Why: Declaring Page entries alone does not allocate visible tab-bar space or place their contents.

Complete captured source

[@gui-boundary/include/gui/contract.hpp](#) | [Lines 124-141](#) | SHA-256 [f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00ffd9db629a404d40da8](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
124 struct Snapshot {
125     std::uint64_t revision=0;
126     std::string title;
127     Size client_size{640,480};
128     double display_scale=1;
129     PixelFormat bitmap_format=PixelFormat::rgb24;
130     std::vector<Page> pages;
131     std::optional<std::string> active_page;
132     // Parents precede their children. Bounds are absolute logical client units.
133     std::vector<Widget> widgets;
134     // Shared chrome placement, in the same coordinates as every widget.
135     Rect page_bar;
136     // A modal group limits input/focus to its subtree; background may remain
137     // visible. Children render in declaration order, later siblings on top.
138     std::optional<WidgetKey> modal_root;
139     std::vector<KeyBinding> key_bindings;
140     Palette palette;
141 };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

12 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	std::uint64_t revision=0;	L125
0: statement	std::string title;	L126
0: statement	Size client_size{640,480};	L127
0: statement	double display_scale=1;	L128
0: statement	PixelFormat bitmap_format=PixelFormat::rgb24;	L129
0: statement	std::vector<Page> pages;	L130
0: statement	std::optional<std::string> active_page;	L131
0: statement	std::vector<Widget> widgets;	L133
0: statement	Rect page_bar;	L135
0: statement	std::optional<WidgetKey> modal_root;	L138
0: statement	std::vector<KeyBinding> key_bindings;	L139
0: statement	Palette palette;	L140

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::Page

struct | @gui-boundary/include/gui/contract.hpp:113 | Lines 113-117 | [Interactive symbol](#) / [full call lists](#)

Owner: gui

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/contract.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing type_identifier), line 330 (missing type_identifier), line 371 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

```
struct Page
```

Why this implementation is on an edit path

[tabs](#) / N1: 1. [Declare pages and selection \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: The tab registry and selection belong to the shared Snapshot. This app currently initializes neither.

Owning type: 1 indexed members in this declaration file. Members not recognized by the syntax scanner remain visible in the complete source below.

Indexed member	Kind / line	Signature
gui::Page::operator==	method L116	bool operator==(const Page&) const = default;

Complete captured source

[@gui-boundary/include/gui/contract.hpp](#) | Lines 113-117 | SHA-256 [f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00ffd9db629a404d40da8](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
113 | struct Page {
114 |     std::string id, label;
115 |     bool enabled=true, visible=true;
116 |     bool operator==(const Page&) const = default;
117 | };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

2 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	std::string id,label;	L114
0: statement	bool enabled=true,visible=true;	L115

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

Example::Example

method | [@gui-boundary/examples/application.hpp:12](#) | Lines 12-45 | [Interactive symbol / full call lists](#)

Owner: Example

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/examples/application.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 132 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

explicit Example(gui::Adapter& adapter):adapter_(adapter)

Why this implementation is on an edit path

[tabs / N1: 1. Declare pages and selection \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: The tab registry and selection belong to the shared Snapshot. This app currently initializes neither.

Complete captured source

[@gui-boundary/examples/application.hpp](#) | Lines 12-45 | SHA-256 [45260f74dcf1d5c6c24514130d58c197d70711d6e06bab283832ce1fd0b3cbf4](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
12     explicit Example(gui::Adapter& adapter):adapter_(adapter) {
13         view_.title="Boundary Workshop";
14         view_.pages={{{"main","Controls"},{"other","Other page"}};view_.active_page="main";
15         add("panel",gui::Kind::group,{});
16         add("heading",gui::Kind::label,{"panel").state.text="Shared controls";
17         auto& choice=add("choice",gui::Kind::choice,{"panel");
18         choice.state.options={{{"first","First","",true},{"second","Second","",true}};
19         choice.state.selected="first";
20         auto& editor=add("editor",gui::Kind::text,{"panel");
21         editor.spec.text_policy={false,false,256,gui::SubmitKey::enter};editor.state.text="Example text";
22         editor.state.options={{{"short","Short text","Hello",true},{"empty","Empty text","",true}};
```

```
23         add("toggle",gui::Kind::toggle,{}, "panel").state.label="Enable action";
24         add("button",gui::Kind::button,{}, "panel").state.label="Add row";
25         auto& menu=add("menu",gui::Kind::menu,{}, "panel");
26         menu.state.label="Actions";
27         menu.state.options={{"clear", "Clear rows", "", true}};
28         auto& list=add("list",gui::Kind::list,{}, "panel");
29         list.spec.follow_tail=true;list.state.placeholder="No rows";
30         list.state.content_size.width=360;
31         auto& bitmap=add("bitmap",gui::Kind::bitmap,{440,48,160,160});
32         bitmap.spec.pointer_input=true;bitmap.state.accessible_name="Color rectangle";
33         bitmap.state.actions={{"refresh", "Refresh image", "", true}};
34         bitmap.state.bitmap={"rectangle",0,gui::solid_bitmap(64,128,192)};
35         add("caption",gui::Kind::label,{440,216,160,48}).state.text="A bitmap area";
36         add("details",gui::Kind::button,{440,280,160,32}).state.label="Show details";
37         auto& overlay=add("dialog",gui::Kind::group,{112,96,416,240});overlay.state.visible=false;
38         add("dialog-title",gui::Kind::label,{136,112,368,32}, "dialog").state.text="A shared modal view";
39         auto& explanation=add("dialog-body",gui::Kind::label,{136,156,368,80}, "dialog");
40         explanation.state.text="This view uses the same declarations in every backend. Escape closes it; background
+         controls cannot receive input.";
41         explanation.state.wrap=gui::TextWrap::word;
42         add("dismiss",gui::Kind::button,{336,272,168,32}, "dialog").state.label="Close details";
43         view_.key_bindings.push_back({gui::ShortcutKey::escape,{ "dismiss",1}});
44         publish();
45     }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 18 detected call sites grouped into 4 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
add L15, L16, L17, L20, L23, L24, L25, L28, L31, L35, L36, L37, L38, L39, L42	foundation::ui::Application::add (gui/shared/application.cpp:29) foundation::Store::add (src/store.cpp:33) Example::add (@gui-boundary/examples/application.hpp:132) +3 additional candidate targets omitted; all targets	ambiguous-name
publish L44	Example::publish (@gui-boundary/examples/application.hpp:150) foundation::ui::Application::publish (gui/shared/application.cpp:300) anonymous namespace @5::App::publish (gui/tests/bezel_test.cpp:39)	ambiguous-name
gui::solid_bitmap L34	No indexed target; external / unresolved.	unresolved

Call name / call sites	Candidate target names / captured locations	Analysis
view_.key_bindings.push_back L43	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

38 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: initialization	:adapter_(adapter)	L12
0: statement	view_.title="Boundary Workshop";	L13
0: statement	view_.pages={{ "main", "Controls"}, {"other", "Other page"} };	L14
0: statement	view_.active_page="main";	L14
0: statement	add("panel", gui::Kind::group, {});	L15
0: statement	add("heading", gui::Kind::label, {}, "panel").state.text="Shared controls";	L16
0: statement	auto& choice=add("choice", gui::Kind::choice, {}, "panel");	L17
0: statement	choice.state.options={{ "first", "First", "", true }, {"second", "Second", "", true } };	L18
0: statement	choice.state.selected="first";	L19
0: statement	auto& editor=add("editor", gui::Kind::text, {}, "panel");	L20
0: statement	editor.spec.text_policy={false, false, 256, gui::SubmitKey::enter};	L21
0: statement	editor.state.text="Example text";	L21
0: statement	editor.state.options={{ "short", "Short text", "Hello", true }, {"empty", "Empty text", "", true } };	L22
0: statement	add("toggle", gui::Kind::toggle, {}, "panel").state.label="Enable action";	L23
0: statement	add("button", gui::Kind::button, {}, "panel").state.label="Add row";	L24
0: statement	auto& menu=add("menu", gui::Kind::menu, {}, "panel");	L25
0: statement	menu.state.label="Actions";	L26
0: statement	menu.state.options={{ "clear", "Clear rows", "", true } };	L27
0: statement	auto& list=add("list", gui::Kind::list, {}, "panel");	L28
0: statement	list.spec.follow_tail=true;	L29
0: statement	list.state.placeholder="No rows";	L29
0: statement	list.state.content_size.width=360;	L30
0: statement	auto& bitmap=add("bitmap", gui::Kind::bitmap, {440, 48, 160, 160});	L31

Depth / structure	Captured label	Source
0: statement	bitmap.spec.pointer_input=true;	L32
0: statement	bitmap.state.accessible_name="Color rectangle";	L32
0: statement	bitmap.state.actions={{"refresh","Refresh image","",true}};	L33
0: statement	bitmap.state.bitmap={"rectangle",0,gui::solid_bitmap(64,128,192)};	L34
0: statement	add("caption",gui::Kind::label,{440,216,160,48}).state.text="A bitmap area";	L35
0: statement	add("details",gui::Kind::button,{440,280,160,32}).state.label="Show details";	L36
0: statement	auto& overlay=add("dialog",gui::Kind::group,{112,96,416,240});	L37
0: statement	overlay.state.visible=false;	L37
0: statement	add("dialog-title",gui::Kind::label,{136,112,368,32},"dialog").state.text="A shared modal view";	L38
0: statement	auto& explanation=add("dialog-body",gui::Kind::label,{136,156,368,80},"dialog");	L39
0: statement	explanation.state.text="This view uses the same declarations in every backend. Escape closes it; background controls cannot receive input.";	L40
0: statement	explanation.state.wrap=gui::TextWrap::word;	L41
0: statement	add("dismiss",gui::Kind::button,{336,272,168,32},"dialog").state.label="Close details";	L42
0: statement	view._key_bindings.push_back({gui::ShortcutKey::escape,{"dismiss",1}});	L43
0: statement	publish();	L44

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

Example::handle

method | [@gui-boundary/examples/application.hpp:67](#) | Lines 67-123 | [Interactive symbol](#) / [full call lists](#)

Owner: Example

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/examples/application.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 132 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

```
void handle(gui::Event event)
```

Why this implementation is on an edit path

[tabs / N5: 5. Handle a page switch \(EDIT\)](#)

Why: The current handler has no PageEvent branch. Normalization already rejects unavailable pages, the active page, and page switches while a modal root is active.

Complete captured source

@gui-boundary/examples/application.hpp | Lines 67-123 | SHA-256 45260f74dcf1d5c6c24514130d58c197d70711d6e06bab283832ce1fd0b3cbf4

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```

67     void handle(gui::Event event) {
68         if(adapter_.closed())return;
69         // Closing must make progress even when measurement or painting fails.
70         if(std::holds_alternative<gui::CloseEvent>(event)) {
71             services_.shutdown();presentation_pending_=false;adapter_.close();return;
72         }
73         if(!std::holds_alternative<gui::ResizeEvent>(event))retry_presentation();
74         // Reuse the same input policy against authoritative state. This also
75         // protects delivery through a queued facade after the view changes.
76         if(!gui::normalize_event(view_,event,[&](const gui::WidgetKey& key) {
77             return adapter_.scroll_offset(key);
78         })))return;
79         if(const auto* e=std::get_if<gui::WidgetEvent>(&event)) {
80             const auto previous_modal=view_.modal_root;
81             const auto available=adapter_.resolved_availability(e->target);
82             auto& w=get(e->target.id);
83             std::visit([&](const auto& input) {
84                 using T=std::decay_t<decltype(input)>;
85                 if constexpr(std::is_same_v<T,gui::SetChecked>)w.state.checked=input.value;
86                 else if constexpr(std::is_same_v<T,gui::EditText>)w.state.text=input.value;
87                 else if constexpr(std::is_same_v<T,gui::ChooseOption>) {
88                     for(const auto& option:w.state.options)if(option.id==input.id) {
89                         if(w.spec.kind==gui::Kind::text)w.state.text=option.value;
90                         else if(w.spec.kind==gui::Kind::choice)w.state.selected=option.id;
91                         else if(w.spec.kind==gui::Kind::menu)get("list").state.records.clear();
92                     }
93                 } else if constexpr(std::is_same_v<T,gui::Activate>) {
94                     if(w.spec.key.id=="details") {
95                         previous_focus_=adapter_.focused();get("dialog").state.visible=true;view_.modal_root=gui::WidgetKey
+ {"dialog",1};
96                     } else if(w.spec.key.id=="dismiss") {
97                         get("dialog").state.visible=false;view_.modal_root.reset();
98                     } else add_row();
99                 }
100             } else if constexpr(std::is_same_v<T,gui::SubmitText>) {
101                 gui::ServiceRequest request;request.id=next_service_++;request.kind=gui::ServiceKind::prompt;
102                 request.title="Enter text";request.byte_limit=80;
103                 if(!services_.enqueue(std::move(request)))throw std::runtime_error("Service request rejected");

```

```
104         } else if constexpr(std::is_same_v<T,gui::SelectRecord>||std::is_same_v<T,gui::ActivateRecord>)w.state.sele
+         cted=input.id;
105         else if constexpr(std::is_same_v<T,gui::InvokeAction>) {
106             if(input.id=="refresh")++w.state.bitmap.revision;
107         } else if constexpr(std::is_same_v<T,gui::PointerInput>) {
108             const auto grid=gui::device_rect(available.bounds,view_.display_scale);
109             if(const auto pixel=gui::pixel_at(available.bounds,grid.width,grid.height,input.position))
110                 get("caption").state.text="Pixel "+std::to_string(unsigned(pixel->x))+",
+             "+std::to_string(unsigned(pixel->y));
111         }
112         },e->input);
113         publish();
114         if(view_.modal_root!=previous_modal) {
115             if(view_.modal_root)adapter_.focus(gui::WidgetKey{"dismiss",1});
116             else {adapter_.focus(previous_focus_);previous_focus_.reset();}
117         }
118     } else if(const auto* page=std::get_if<gui::PageEvent>(&event)) {
119         view_.active_page=page->id;publish();
120     } else if(const auto* resize=std::get_if<gui::ResizeEvent>(&event)) {
121         view_.client_size=resize->client_size;view_.display_scale=resize->display_scale;publish();
122     }
123 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 55 indexed candidate call sites across 30 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::host::Browser::Browser::anonymous lambda @14 (gui/host/browser.hpp:14)	L15	ambiguous-name
foundation::host (gui/host/contract.hpp:7)	L13	ambiguous-name
foundation::host::Session::Session::anonymous lambda @45 (gui/host/contract.hpp:45)	L46	ambiguous-name
foundation::host::finish_smoke (gui/host/qualification.hpp:12)	L13	ambiguous-name
main (gui/hosts/framebuffer_main.cpp:10)	L40	ambiguous-name
main::anonymous lambda @26 (gui/hosts/framebuffer_main.cpp:26)	L27	ambiguous-name
foundation::ui::Application::qualify (gui/shared/application.cpp:177)	L203, L210, L212, L214	ambiguous-name
foundation::ui::Application::qualify::edit (gui/shared/application.cpp:186)	L187	ambiguous-name

Print scope: 22 additional caller symbols containing 44 candidate call sites are omitted. Map-linked and same-file callers are listed first. [All callers in explorer](#)

Outgoing: 19 detected call sites grouped into 16 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
publish L113 , L119 , L121	Example::publish (@gui-boundary/examples/application.hpp:150) foundation::ui::Application::publish (gui/shared/application.cpp:300) anonymous namespace @5::App::publish (gui/tests/bezel_test.cpp:39)	ambiguous-name
services_.shutdown L71	foundation::ui::Application::shutdown (gui/shared/application.cpp:266) foundation::host::AdapterServices::shutdown (gui/host/contract.hpp:34) foundation::host::FileServices::shutdown (gui/host/file_services.hpp:164) +6 additional candidate targets omitted ; all targets	ambiguous-name
gui::normalize_event L76	gui::normalize_event (@gui-boundary/include/gui/contract.hpp:371)	name-candidate
retry_presentation L73	Example::retry_presentation (@gui-boundary/examples/application.hpp:61) foundation::ui::Application::retry_presentation (gui/shared/application.cpp:296) anonymous namespace @5::App::retry_presentation (gui/tests/bezel_test.cpp:62) +1 additional candidate targets omitted ; all targets	ambiguous-name
get L82	Example::get (@gui-boundary/examples/application.hpp:141) foundation::ui::Application::get (gui/shared/application.hpp:58) foundation::Store::get (src/store.cpp:65)	ambiguous-name
adapter_.closed L68	No indexed target; external / unresolved.	unresolved
std::holds_alternative<gui::CloseEvent> L70	No indexed target; external / unresolved.	unresolved
adapter_.close L71	foundation::host::file_detail::File::close (gui/host/file_services.hpp:34) anonymous namespace @18::Context::close (tools/windows_gl_probe.cpp:143)	ambiguous-name

Print scope: 8 additional outgoing groups containing 9 call sites are omitted. Groups containing map-linked or same-file candidates are listed first. [All outgoing calls in explorer](#)

Complete indexed structural outline

41 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(adapter_.closed())	L68
1: then	then	L68
2: return	return;	L68
0: branch	(std::holds_alternative<gui::CloseEvent>(event))	L70
1: then	then	L70
2: statement	services_.shutdown() ;	L71
2: statement	presentation_pending_=false ;	L71

Depth / structure	Captured label	Source
2: statement	adapter_.close();	L71
2: return	return;	L71
0: branch	(!std::holds_alternative<gui::ResizeEvent>(event))	L73
1: then	then	L73
2: statement	retry_presentation();	L73
0: branch	(!gui::normalize_event(view_.event,&)(const gui::WidgetKey& key) { return adapter_.scroll_offset(key); })))	L76
1: then	then	L78
2: return	return;	L78
0: branch	(const auto* e=std::get_if<gui::WidgetEvent>(&event))	L79
1: then	then	L79
2: statement	const auto previous_modal=view_.modal_root;	L80
2: statement	const auto available=adapter_.resolved_availability(e->target);	L81
2: statement	auto& w=get(e->target.id);	L82
2: statement	std::visit([&](const auto& input) { using T=std::decay_t<decltype(input)>; if constexpr(std::is_same_v<T,gui::SetChecked>)w.state.checked=input.value; else if constexpr(std::is_sa...	L83
2: statement	publish();	L113
2: branch	(view_.modal_root!=previous_modal)	L114
3: then	then	L114
4: branch	(view_.modal_root)	L115
5: then	then	L115
6: statement	adapter_.focus(gui::WidgetKey{"dismiss",1});	L115
5: else	else	L116
6: statement	adapter_.focus(previous_focus_);	L116
6: statement	previous_focus_.reset();	L116
1: else	else	L118
2: branch	(const auto* page=std::get_if<gui::PageEvent>(&event))	L118
3: then	then	L118
4: statement	view_.active_page=page->id;	L119
4: statement	publish();	L119

Depth / structure	Captured label	Source
3: else	else	L120
4: branch	(const auto* resize=std::get_if<gui::ResizeEvent>(&event))	L120
5: then	then	L120
6: statement	view_.client_size=resize->client_size;	L121
6: statement	view_.display_scale=resize->display_scale;	L121
6: statement	publish();	L121

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::page_tabs

function | [@gui-boundary/include/gui/presentation.hpp:35](#) | Lines 35-44 | [Interactive symbol / full call lists](#)

Owner: gui

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/presentation.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Captured signature

```
inline std::vector<PageTab> page_tabs(const Snapshot& snapshot)
```

Why this implementation is on an edit path

[tabs](#) / N6: 6. Reuse tab rendering (REUSE)

Why: Pages are an existing GUI contract capability. Supplier excerpts are evidence and examples, not edit targets.

Complete captured source

[@gui-boundary/include/gui/presentation.hpp](#) | Lines 35-44 | SHA-256 70c593ad1086d99dfec0f1f745e6842a90fcd8ba19801e0774d28d22430608ab

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
35 | inline std::vector<PageTab> page_tabs(const Snapshot& snapshot) {
36 |     std::vector<PageTab> result;
37 |     const auto count=std::count_if(snapshot.pages.begin(),snapshot.pages.end(),[](const Page& p){return p.visible;});
38 |     if(count==0||!has_area(snapshot.page_bar))return result;
39 |     const auto width=snapshot.page_bar.width/double(count);
40 |     for(const auto& page:snapshot.pages)if(page.visible)
41 |         result.push_back({page.id,page.label,{snapshot.page_bar.x+double(result.size())*width,snapshot.page_bar.y,width,sna
+ | pshot.page_bar.height},
42 |         page.enabled&&!snapshot.modal_root,snapshot.active_page==page.id});
43 |     return result;
44 | }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 8 detected call sites grouped into 7 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
<code>std::count_if</code> L37	No indexed target; external / unresolved.	unresolved
<code>snapshot.pages.begin</code> L37	No indexed target; external / unresolved.	unresolved
<code>snapshot.pages.end</code> L37	No indexed target; external / unresolved.	unresolved
<code>has_area</code> L38	gui::has_area (@gui-boundary/include/gui/geometry.hpp:24)	name-candidate
<code>double</code> L39 , L41	No indexed target; external / unresolved.	unresolved
<code>result.push_back</code> L41	No indexed target; external / unresolved.	unresolved
<code>result.size</code> L41	foundation::Store::size (src/store.cpp:73)	name-candidate

Complete indexed structural outline

11 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	<code>std::vector<PageTab> result;</code>	L36
0: statement	<code>const auto count=std::count_if(snapshot.pages.begin(),snapshot.pages.end(),[](const Page& p){return p.visible;});</code>	L37
0: branch	<code>(count==0 !has_area(snapshot.page_bar))</code>	L38
1: then	<code>then</code>	L38
2: return	<code>return result;</code>	L38
0: statement	<code>const auto width=snapshot.page_bar.width/double(count);</code>	L39
0: loop	<code>for(const auto& page:snapshot.pages)</code>	L40

Depth / structure	Captured label	Source
1: branch	(page.visible)	L40
2: then	then	L41
3: statement	result.push_back({page.id,page.label,{snapshot.page_bar.x+double(result.size())*width,snapshot.page_bar.y,width,snapshot.page_bar.height}, page.enabled&&!snapshot.modal_root,snap...	L41
0: return	return result;	L43

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::Adapter

class | [@gui-boundary/include/gui/contract.hpp:396](#) | Lines 396-414 | [Interactive symbol / full call lists](#)

Owner: gui

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/contract.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing type_identifier), line 330 (missing type_identifier), line 371 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

class Adapter

Why this implementation is on an edit path

[tabs / N7: 7. Reuse adapter focus handling \(REUSE\)](#)

Why: The existing adapter owns focus and availability. Add application focus policy only when the feature needs it.

Owning type: 1 indexed members in this declaration file. Members not recognized by the syntax scanner remain visible in the complete source below.

Indexed member	Kind / line	Signature
gui::Adapter::~~Adapter	method L398	virtual ~Adapter()=default;

Complete captured source

[@gui-boundary/include/gui/contract.hpp](#) | Lines 396-414 | SHA-256 [f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00ffd9db629a404d40da8](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
396 | class Adapter {
397 | public:
398 |     virtual ~Adapter()=default;
399 |     virtual void present(Snapshot snapshot)=0;
400 |     virtual Size measure_text(const TextMeasureRequest& request) const=0;
```

```
401     virtual Availability resolved_availability(const WidgetKey& target) const=0;
402     virtual std::optional<WidgetKey> focused() const=0;
403     virtual bool focus(std::optional<WidgetKey> target)=0;
404     virtual bool focus_next(bool reverse=false)=0;
405     virtual void scroll(const WidgetKey& target,Point offset)=0;
406     virtual Point scroll_offset(const WidgetKey& target) const=0;
407     virtual TextSelection text_selection(const WidgetKey& target) const=0;
408     virtual void text_selection(const WidgetKey& target,TextSelection selection)=0;
409     virtual bool open_popup(const WidgetKey& target)=0;
410     virtual void close_popup(const WidgetKey& target)=0;
411     virtual void invalidate(const WidgetKey& target,PixelRect damage)=0;
412     virtual bool closed() const=0;
413     virtual void close();
414 };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

16 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	public	L397
0: statement	virtual void present(Snapshot snapshot)=0;	L399
0: statement	virtual Size measure_text(const TextMeasureRequest& request) const=0;	L400
0: statement	virtual Availability resolved_availability(const WidgetKey& target) const=0;	L401
0: statement	virtual std::optional<WidgetKey> focused() const=0;	L402
0: statement	virtual bool focus(std::optional<WidgetKey> target)=0;	L403
0: statement	virtual bool focus_next(bool reverse=false)=0;	L404
0: statement	virtual void scroll(const WidgetKey& target,Point offset)=0;	L405
0: statement	virtual Point scroll_offset(const WidgetKey& target) const=0;	L406
0: statement	virtual TextSelection text_selection(const WidgetKey& target) const=0;	L407

Depth / structure	Captured label	Source
0: statement	virtual void text_selection(const WidgetKey& target,TextSelection selection)=0;	L408
0: statement	virtual bool open_popup(const WidgetKey& target)=0;	L409
0: statement	virtual void close_popup(const WidgetKey& target)=0;	L410
0: statement	virtual void invalidate(const WidgetKey& target,PixelRect damage)=0;	L411
0: statement	virtual bool closed() const=0;	L412
0: statement	virtual void close();=0;	L413

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

_metadata

[function](#) | [tools/rust_build.py:335](#) | [Lines 335-357](#) | [Interactive symbol](#) / [full call lists](#)

Captured signature

```
def _metadata(root, cargo, environment, build):
```

Why this implementation is on an edit path

[source-files / N6: Rust module .rs \(EDIT\)](#)

Why: A new crate, dependency, or build.rs is a separate build-policy change, not this internal-module recipe.

Complete captured source

[tools/rust_build.py](#) | [Lines 335-357](#) | [SHA-256 7876c40716bf1fcc596caa5551e2fc169ae106580ab7479b4f002a6d7843dfa5](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
335 def _metadata(root, cargo, environment, build):
336     raw = _run([cargo, 'metadata', '--frozen', '--no-deps', '--format-version', '1',
337               '--manifest-path', str(root / 'rust/Cargo.toml')],
338               cwd=build, environment=environment)
339     data = json.loads(raw)
340     packages = data.get('packages', [])
341     if (len(packages) != 1 or len(data.get('workspace_members', [])) != 1
342         or Path(data.get('workspace_root', '')).resolve() != root / 'rust'):
343         raise ValueError('Rust pilot requires its one dependency-free workspace member')
344     package = packages[0]
345     expected = root / 'rust/text_validation/Cargo.toml'
346     if (package.get('name') != 'foundation-text-validation'
347         or Path(package.get('manifest_path', '')).resolve() != expected or package.get('source') is not None
348         or package.get('dependencies') or package.get('edition') != '2021'
349         or package.get('rust_version') != '1.63'):
```

```
350         raise ValueError('Rust pilot requires edition 2021, Rust 1.63 and zero external crates')
351     targets = package.get('targets', [])
352     if (len(targets) != 1 or targets[0].get('name') != 'foundation_rust'
353         or set(targets[0].get('crate_types', [])) != {'staticlib', 'rlib'}
354         or set(targets[0].get('kind', [])) != {'staticlib', 'rlib'}):
355         raise ValueError('Rust pilot requires only the foundation_rust staticlib and rlib; no build scripts or macros')
356     return {'package_id': package['id'], 'workspace_root': data['workspace_root'],
357           'manifest_path': str(expected)}
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 3 indexed candidate call sites across 3 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
describe (tools/rust_build.py:440)	L495	name-candidate
PublicGitHub.json (tools/distro_client.py:136)	L143	ambiguous-name
PublicGitHub.pages (tools/distro_client.py:145)	L156	ambiguous-name

Outgoing: 29 detected call sites grouped into 12 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
_run L336	_run (tools/rust_build.py:162)	name-candidate
str L337, L357	No indexed target; external / unresolved.	unresolved
json.loads L339	No indexed target; external / unresolved.	unresolved
data.get L340, L341, L342	Browser.get (tests/test_gallery_browser.py:57)	name-candidate
len L341, L352	No indexed target; external / unresolved.	unresolved
Path(data.get('workspace_root', '')).resolve L342	NativeCheckTests.test_gentoo_runtime_preflight_configures_only_its_required_provider.resolve (tests/test_distro_check.py:109) CoverageTests.test_windows_drive_case_normalizes_only_known_roots.LexicalWindowsPath.resolve (tests/test_test_plan.py:113)	ambiguous-name
Path L342, L347	No indexed target; external / unresolved.	unresolved

Call name / call sites	Candidate target names / captured locations	Analysis
ValueError L343, L350, L355	No indexed target; external / unresolved.	unresolved

Print scope: 4 additional outgoing groups containing 13 call sites are omitted. Groups containing map-linked or same-file candidates are listed first. [All outgoing calls in explorer](#)

Complete indexed structural outline

16 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	raw = _run([cargo, 'metadata', '--frozen', '--no-deps', '--format-version', '1', '--manifest-path', str(root / 'rust/Cargo.toml')], cwd=build, environment=environment)	L336
0: statement	data = json.loads(raw)	L339
0: statement	packages = data.get('packages', [])	L340
0: branch	(len(packages) != 1 or len(data.get('workspace_members', [])) != 1 or Path(data.get('workspace_root', "")).resolve() != root / 'rust')	L341
1: then	then	L343
2: throw	raise ValueError('Rust pilot requires its one dependency-free workspace member')	L343
0: statement	package = packages[0]	L344
0: statement	expected = root / 'rust/text_validation/Cargo.toml'	L345
0: branch	(package.get('name') != 'foundation-text-validation' or Path(package.get('manifest_path', "")).resolve() != expected or package.get('source') is not None or package.get('dependenc...	L346
1: then	then	L350
2: throw	raise ValueError('Rust pilot requires edition 2021, Rust 1.63 and zero external crates')	L350
0: statement	targets = package.get('targets', [])	L351
0: branch	(len(targets) != 1 or targets[0].get('name') != 'foundation_rust' or set(targets[0].get('crate_types', [])) != {'staticlib', 'rlib'} or set(targets[0].get('kind', [])) != {'static...	L352
1: then	then	L355
2: throw	raise ValueError('Rust pilot requires only the foundation_rust staticlib and rlib; no build scripts or macros')	L355
0: return	return {'package_id': package['id'], 'workspace_root': data['workspace_root'], 'manifest_path': str(expected)}	L356

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::Store::validate

method | [src/store.cpp:17](#) | [Lines 17-31](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::Store

Captured signature

```
void Store::validate(std::string_view text)
```

Why this implementation is on an edit path

validation / N1: 1. Identify the rule owner (DECISION)

Why: CLI, GUI and import should receive the same rule through core; a UI-only filter would leave other callers different.

validation / N4: 4. Apply rules before mutation (EDIT)

Why: Returning an old ASCII-only message for a different rule would mislead the caller.

widget-runtime / N7: Store::validate (REUSE)

Why: This is a concrete function called by the function beneath the event.

Complete captured source

[src/store.cpp](#) | [Lines 17-31](#) | [SHA-256 59580ea4cdb305bba9c97c8faaccbf2c390b55893a40785272a3f7a72888a0c3](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
17 void Store::validate(std::string_view text) {
18     static_assert(max_text_bytes == 256, "private text ABI length limit changed");
19     const auto status = detail::validate_text(
20         reinterpret_cast<const std::uint8_t*>(text.data()), text.size());
21     switch (status) {
22     case FOUNDATION_TEXT_OK:
23         return;
24     case FOUNDATION_TEXT_INVALID_LENGTH:
25         throw std::invalid_argument("text must contain 1..256 bytes");
26     case FOUNDATION_TEXT_INVALID_ASCII:
27         throw std::invalid_argument("text must contain printable ASCII only");
28     default:
29         throw std::logic_error("text validation component failed");
30     }
31 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 2 indexed candidate call sites across 2 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::Store::add (src/store.cpp:33)	L34	name-candidate
foundation::Store::update (src/store.cpp:47)	L48	name-candidate

Outgoing: 7 detected call sites grouped into 6 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
detail::validate_text L19	validate_text (rust/text_validation/src/lib.rs:11) foundation::detail::validate_text (src/text_validation.cpp:32) foundation::detail::validate_text (tests/rust_component_status_test.cpp:29)	ambiguous-name
text.size L20	foundation::Store::size (src/store.cpp:73)	name-candidate
reinterpret_cast<const std::uint8_t*> L20	No indexed target; external / unresolved.	unresolved
text.data L20	No indexed target; external / unresolved.	unresolved
std::invalid_argument L25, L27	No indexed target; external / unresolved.	unresolved
std::logic_error L29	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

11 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	static_assert(max_text_bytes == 256, "private text ABI length limit changed");	L18
0: statement	const auto status = detail::validate_text(reinterpret_cast<const std::uint8_t*>(text.data()), text.size());	L19
0: switch	(status)	L21
1: case	FOUNDATION_TEXT_OK	L22
2: return	return;	L23
1: case	FOUNDATION_TEXT_INVALID_LENGTH	L24
2: throw	throw std::invalid_argument("text must contain 1..256 bytes");	L25
1: case	FOUNDATION_TEXT_INVALID_ASCII	L26
2: throw	throw std::invalid_argument("text must contain printable ASCII only");	L27
1: case	default	L28
2: throw	throw std::logic_error("text validation component failed");	L29

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

validate_text

[function](#) | [rust/text_validation/src/lib.rs:11](#) | [Lines 11-19](#) | [Interactive symbol](#) / [full call lists](#)

Captured signature

fn validate_text(bytes: &[u8]) -> u32

Why this implementation is on an edit path

[validation](#) / [N3: 3. Match both text providers \(EDIT\)](#)

Why: Fresh configurations use Rust; selecting C++ must retain equivalent accepted input and status meaning.

[widget-runtime](#) / [N6: Selected text validator \(REUSE\)](#)

Why: Follow these deeper links when changing validation itself.

Complete captured source

[rust/text_validation/src/lib.rs](#) | [Lines 11-19](#) | [SHA-256 40e5e80c7fde0da00914099ce161ab3c1da1ae80c42d290c9f7cdf397373693d](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
11 fn validate_text(bytes: &[u8]) -> u32 {
12     if bytes.is_empty() || bytes.len() > MAX_TEXT_BYTES {
13         return INVALID_LENGTH;
14     }
15     if bytes.iter().any(|&byte| byte < 32 || byte > 126) {
16         return INVALID_ASCII;
17     }
18     OK
19 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 3 indexed candidate call sites across 3 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation_text_validate_v1 (rust/text_validation/src/lib.rs:28)	L38	name-candidate
foundation::Store::validate (src/store.cpp:17)	L19	ambiguous-name
main (tests/rust_component_test.cpp:14)	L24	ambiguous-name

Outgoing: 4 detected call sites grouped into 4 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
<code>bytes.is_empty</code> L12	No indexed target; external / unresolved.	unresolved
<code>bytes.len</code> L12	No indexed target; external / unresolved.	unresolved
<code>bytes.iter().any</code> L15	No indexed target; external / unresolved.	unresolved
<code>bytes.iter</code> L15	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

11 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	if bytes.is_empty() bytes.len() > MAX_TEXT_BYTES { return INVALID_LENGTH; }	L12
1: branch	bytes.is_empty() bytes.len() > MAX_TEXT_BYTES	L12
2: then	then	L12
3: statement	return INVALID_LENGTH;	L13
4: return	return INVALID_LENGTH	L13
0: statement	if bytes.iter().any(&byte byte < 32 byte > 126) { return INVALID_ASCII; }	L15
1: branch	bytes.iter().any(&byte byte < 32 byte > 126)	L15
2: then	then	L15
3: statement	return INVALID_ASCII;	L16
4: return	return INVALID_ASCII	L16
0: statement	OK	L18

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation_text_validate_v1

function | [src/text_validation.cpp:14](#) | [Lines 14-24](#) | [Interactive symbol](#) / [full call lists](#)

Entry-point clue: Declared with C linkage; foreign-call/export entry candidate.

Scanner caveats for this file:

Syntax parser recovered from errors: line 6 (ERROR), line 9 (ERROR). Recovered structure may be incomplete.

Captured signature

```
uint32_t foundation_text_validate_v1(const uint8_t *bytes, size_t length) noexcept
```

Why this implementation is on an edit path

[validation](#) / N3: [3. Match both text providers \(EDIT\)](#)

Why: Fresh configurations use Rust; selecting C++ must retain equivalent accepted input and status meaning.

[widget-runtime](#) / N6: [Selected text validator \(REUSE\)](#)

Why: Follow these deeper links when changing validation itself.

Complete captured source

[src/text_validation.cpp](#) | [Lines 14-24](#) | [SHA-256 509ec58625f832343e9e762e0b8dbf1df2519c67f2bf52839a1a9b990b2507bb](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
14 | extern "C" uint32_t foundation_text_validate_v1(const uint8_t *bytes,
15 |                                     size_t length) noexcept {
16 |     if (length == 0 || length > 256) return FOUNDATION_TEXT_INVALID_LENGTH;
17 |     if (bytes == nullptr) return FOUNDATION_TEXT_INVALID_POINTER;
18 |     for (size_t index = 0; index < length; ++index) {
19 |         if (bytes[index] < 32 || bytes[index] > 126) {
20 |             return FOUNDATION_TEXT_INVALID_ASCII;
21 |         }
22 |     }
23 |     return FOUNDATION_TEXT_OK;
24 | }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 6 indexed candidate call sites across 2 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::detail::validate_text (src/text_validation.cpp:32)	L33	name-candidate
main (tests/rust_component_test.cpp:14)	L29, L34, L41, L48, L57	ambiguous-name

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.
No calls detected in this captured symbol.

Complete indexed structural outline

11 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(length == 0 length > 256)	L16
1: then	then	L16
2: return	return FOUNDATION_TEXT_INVALID_LENGTH;	L16
0: branch	(bytes == nullptr)	L17
1: then	then	L17
2: return	return FOUNDATION_TEXT_INVALID_POINTER;	L17
0: loop	for (size_t index = 0; index < length; ++index)	L18
1: branch	(bytes[index] < 32 bytes[index] > 126)	L19
2: then	then	L19
3: return	return FOUNDATION_TEXT_INVALID_ASCII;	L20
0: return	return FOUNDATION_TEXT_OK;	L23

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

anonymous enum @14

enum | [src/text_validation.h:14](#) | Lines 14-21 | [Interactive symbol / full call lists](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 1 (ERROR), line 33 (ERROR), line 35 (ERROR), line 38 (ERROR), line 40 (ERROR). Recovered structure may be incomplete.

Captured signature

enum

Why this implementation is on an edit path

[validation / N3: 3. Match both text providers \(EDIT\)](#)

Why: Fresh configurations use Rust; selecting C++ must retain equivalent accepted input and status meaning.

Complete captured source

[src/text_validation.h](#) | Lines 14-21 | SHA-256 3bf3c4e63fc055fc67fab79c53e67f1c70c6cf4d3282aa502194b9939df08805

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
14  enum {
15      FOUNDATION_TEXT_OK = 0,
16      FOUNDATION_TEXT_INVALID_LENGTH = 1,
17      FOUNDATION_TEXT_INVALID_ASCII = 2,
18      FOUNDATION_TEXT_INVALID_POINTER = 3,
19      FOUNDATION_TEXT_PROVIDER_CPP = 0,
20      FOUNDATION_TEXT_PROVIDER_RUST = 1
21  };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

1 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	{ FOUNDATION_TEXT_OK = 0, FOUNDATION_TEXT_INVALID_LENGTH = 1, FOUNDATION_TEXT_INVALID_ASCII = 2, FOUNDATION_TEXT_INVALID_POINTER = 3, FOUNDATION_TEXT_PROVIDER_CPP = 0, FOUNDATION_...	L14

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::Store::update

method | [src/store.cpp:47](#) | [Lines 47-55](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::Store

Captured signature

bool Store::update(RecordId id, std::string_view text)

Why this implementation is on an edit path

[validation](#) / [N4: 4. Apply rules before mutation \(EDIT\)](#)

Why: Returning an old ASCII-only message for a different rule would mislead the caller.

[core-operation](#) / [N1: 1. Reuse the existing operation \(DECISION\)](#)

Why: The same API can serve a future CLI command; the current CLI has no edit command or persistent store.

core-operation / N4: 4. Call Store::update (REUSE)
Why: update validates text first, returns false for a missing ID when the text is valid, and prepares a replacement string before swapping it into place. The record ID stays unchanged.
core-operation / N5: 5. Handle rejected/missing input (EDIT)
Why: Invalid replacement text can throw before missing-ID lookup, so both outcomes need explicit handling.

Complete captured source

src/store.cpp | Lines 47-55 | SHA-256 59580ea4cdb305bba9c97c8faaccbf2c390b55893a40785272a3f7a72888a0c3
Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
47 | bool Store::update(RecordId id, std::string_view text) {
48 |     validate(text);
49 |     const auto found = std::find_if(records_.begin(), records_.end(),
50 |         [id](const Record& row) { return row.id == id; });
51 |     if (found == records_.end()) return false;
52 |     std::string replacement(text);
53 |     found->text.swap(replacement);
54 |     return true;
55 | }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.
Incoming: 12 indexed candidate call sites across 11 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
main::anonymous lambda @43 (tests/rust_component_status_test.cpp:43)	L43	name-candidate
main::anonymous lambda @50 (tests/rust_component_status_test.cpp:50)	L50	name-candidate
main::anonymous lambda @52 (tests/rust_component_status_test.cpp:52)	L52	name-candidate
main (tests/store_test.cpp:26)	L48, L49	name-candidate
main::anonymous lambda @40 (tests/store_test.cpp:40)	L40	name-candidate
main::anonymous lambda @41 (tests/store_test.cpp:41)	L41	name-candidate
main::anonymous lambda @42 (tests/store_test.cpp:42)	L42	name-candidate
main::anonymous lambda @43 (tests/store_test.cpp:43)	L43	name-candidate

Print scope: 3 additional caller symbols containing 3 candidate call sites are omitted. Map-linked and same-file callers are listed first. [All callers in explorer](#)
Outgoing: 6 detected call sites grouped into 5 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
validate L48	foundation::Store::validate (src/store.cpp:17)	name-candidate
std::find_if L49	No indexed target; external / unresolved.	unresolved
records_.begin L49	No indexed target; external / unresolved.	unresolved
records_.end L49, L51	No indexed target; external / unresolved.	unresolved
found->text.swap L53	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

8 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	validate(text);	L48
0: statement	const auto found = std::find_if(records_.begin(), records_.end(), [id](const Record& row) { return row.id == id; });	L49
0: branch	(found == records_.end())	L51
1: then	then	L51
2: return	return false;	L51
0: statement	std::string replacement(text);	L52
0: statement	found->text.swap(replacement);	L53
0: return	return true;	L54

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::TextTask::start

method | [gui/shared/task.hpp:25](#) | [Lines 25-38](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui::TextTask

Captured signature

TaskUpdate start(std::vector<std::string> input)

Why this implementation is on an edit path

[validation](#) / [N5: 5. Follow mirrored limits \(EDIT\)](#)

Why: The current private text ABI is bounded to 256 bytes, and tasks separately enforce 256-byte records.

Complete captured source

[gui/shared/task.hpp](#) | [Lines 25-38](#) | [SHA-256 407e841598299b41c8a4e726a44341d05dbeb5e5b46b5b0098d8a2b09e10a9fe](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
25     TaskUpdate start(std::vector<std::string> input) {
26         if (closed_) throw std::logic_error("Task executor is closed");
27         if (input.size() > 1000) throw std::length_error("Task input exceeds record limit");
28         std::size_t total = 0;
29         for (const auto& text : input) {
30             if (text.size() > 256) throw std::length_error("Task input exceeds text limit");
31             total += text.size();
32         }
33         if (update_.generation == std::numeric_limits<std::uint64_t>::max())
34             throw std::overflow_error("Task identity exhausted");
35         update_ = {update_.generation + 1, 0, total, 0, false};
36         input_ = std::move(input); row_ = offset_ = 0; running_ = true;
37         return update_;
38     }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 17 indexed candidate call sites across 8 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::ui::CooperativeTaskExecutor::start (gui/shared/task.hpp:69)	L69	ambiguous-name
foundation::host::FileServices::service::anonymous lambda @140 (gui/host/file_services.hpp:140)	L143	ambiguous-name

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::handle::anonymous lambda @93 (gui/shared/application.cpp:93)	L107	ambiguous-name
foundation::ui::anonymous namespace @7::NativeTaskExecutor::run (gui/shared/task_native.cpp:20)	L27	ambiguous-name
foundation::ui::anonymous namespace @7::NativeTaskExecutor::start (gui/shared/task_native.cpp:41)	L43	ambiguous-name
anonymous namespace @5::SlowExecutor::start (gui/tests/bezel_test.cpp:222)	L222	ambiguous-name
main (gui/tests/task_native_test.cpp:21)	L24, L28, L30, L34, L35, L38	ambiguous-name
owned_task (gui/tests/task_test.cpp:13)	L16, L21, L23, L24, L26	ambiguous-name

Outgoing: 9 detected call sites grouped into 7 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
std::logic_error L26	No indexed target; external / unresolved.	unresolved
input.size L27	foundation::Store::size (src/store.cpp:73)	name-candidate
std::length_error L27, L30	No indexed target; external / unresolved.	unresolved
text.size L30, L31	foundation::Store::size (src/store.cpp:73)	name-candidate
std::numeric_limits<std::uint64_t>::max L33	No indexed target; external / unresolved.	unresolved
std::overflow_error L34	No indexed target; external / unresolved.	unresolved
std::move L36	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

20 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(closed_)	L26
1: then	then	L26
2: throw	throw std::logic_error("Task executor is closed");	L26
0: branch	(input.size() > 1000)	L27

Depth / structure	Captured label	Source
1: then	then	L27
2: throw	throw std::length_error("Task input exceeds record limit");	L27
0: statement	std::size_t total = 0;	L28
0: loop	for (const auto& text : input)	L29
1: branch	(text.size() > 256)	L30
2: then	then	L30
3: throw	throw std::length_error("Task input exceeds text limit");	L30
1: statement	total += text.size();	L31
0: branch	(update_generation == std::numeric_limits<std::uint64_t>::max())	L33
1: then	then	L34
2: throw	throw std::overflow_error("Task identity exhausted");	L34
0: statement	update_ = {update_generation + 1, 0, total, 0, false};	L35
0: statement	input_ = std::move(input);	L36
0: statement	row_ = offset_ = 0;	L36
0: statement	running_ = true;	L36
0: return	return update_;	L37

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

anonymous namespace @12::run

function | [src/main.cpp:13](#) | [Lines 13-46](#) | [Interactive symbol](#) / [full call lists](#)

Owner: anonymous namespace @12

Captured signature

```
int run(int argc, const char* const* argv)
```

Why this implementation is on an edit path

[validation](#) / [N6: 6. Follow existing consumers \(REUSE\)](#)

Why: The validation remains one reusable behavior rather than a separate rule per frontend.

[core-operation](#) / [N1: 1. Reuse the existing operation \(DECISION\)](#)

Why: The same API can serve a future CLI command; the current CLI has no edit command or persistent store.

Complete captured source

src/main.cpp | Lines 13-46 | SHA-256 845261d1aaf8b7171c3ea4dee85aa368c8088092d85ab45f55409a848d28e9e8

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
13 int run(int argc, const char* const* argv) {
14     try {
15         if (argc == 2 && std::string_view(argv[1]) == "--version") {
16             std::cout << "software-foundation " << FOUNDATION_VERSION << '\n';
17             return std::cout.flush() ? 0 : 1;
18         }
19         if (argc == 2 && std::string_view(argv[1]) == "--self-check") {
20             foundation::Store store(1);
21             const auto id = store.add("example");
22             return store.get(id)->text == "example" && store.erase(id) ? 0 : 1;
23         }
24         if (argc == 1 || (argc == 2 && std::string_view(argv[1]) == "--help")) {
25             std::cout << "Usage: foundation-cli [--version|--self-check|--help]\n"
26                 << "      foundation-cli -- TEXT [TEXT ...]\n"
27                 << "Print a bounded collection, one record per line.\n";
28             return std::cout.flush() ? 0 : 1;
29         }
30         if (std::string_view(argv[1]) != "--" || argc < 3) {
31             std::cerr << "Use --help for usage.\n";
32             return 2;
33         }
34         foundation::Store store;
35         // Validate the complete request before emitting any records.
36         for (int i = 2; i < argc; ++i) store.add(argv[i]);
37         for (const auto& row : store.snapshot()) {
38             std::cout << row.id << '\t' << row.text << '\n';
39         }
40         if (!std::cout.flush()) return 1;
41     } catch (const std::exception& error) {
42         std::cerr << error.what() << '\n';
43         return 1;
44     }
45     return 0;
46 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 3 indexed candidate call sites across 3 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
wmain (src/main.cpp:50)	L59	ambiguous-name
main (src/main.cpp:66)	L66	ambiguous-name
foundation::ui::anonymous namespace @7::NativeTaskExecutor::anonymous lambda @19 (gui/shared/task_native.cpp:19)	L19	ambiguous-name

Outgoing: 13 detected call sites grouped into 7 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
store.add L21, L36	foundation::ui::Application::add (gui/shared/application.cpp:29) foundation::Store::add (src/store.cpp:33) Example::add (@gui-boundary/examples/application.hpp:132) +3 additional candidate targets omitted; all targets	ambiguous-name
std::string_view L15, L19, L24, L30	No indexed target; external / unresolved.	unresolved
std::cout.flush L17, L28, L40	No indexed target; external / unresolved.	unresolved
store.get L22	Example::get (@gui-boundary/examples/application.hpp:141) foundation::ui::Application::get (gui/shared/application.hpp:58) foundation::Store::get (src/store.cpp:65)	ambiguous-name
store.erase L22	foundation::Store::erase (src/store.cpp:57)	name-candidate
store.snapshot L37	foundation::Store::snapshot (src/store.cpp:72)	name-candidate
error.what L42	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

46 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: try	try	L14
1: try_body	try body	L14
2: branch	(argc == 2 && std::string_view(argv[1]) == "--version")	L15
3: then	then	L15
4: statement	std::cout << "software-foundation " << FOUNDATION_VERSION << '\n';	L16

Depth / structure	Captured label	Source
4: return	return std::cout.flush() ? 0 : 1;	L17
5: branch	std::cout.flush()	L17
6: then	then	L17
7: statement	0	L17
6: else	else	L17
7: statement	1	L17
2: branch	(argc == 2 && std::string_view(argv[1]) == "--self-check")	L19
3: then	then	L19
4: statement	foundation::Store store(1);	L20
4: statement	const auto id = store.add("example");	L21
4: return	return store.get(id)->text == "example" && store.erase(id) ? 0 : 1;	L22
5: branch	store.get(id)->text == "example" && store.erase(id)	L22
6: then	then	L22
7: statement	0	L22
6: else	else	L22
7: statement	1	L22
2: branch	(argc == 1 (argc == 2 && std::string_view(argv[1]) == "--help"))	L24
3: then	then	L24
4: statement	std::cout << "Usage: foundation-cli [--version --self-check --help]\n" " foundation-cli -- TEXT [TEXT ...]\n" "Print a bounded collection, one record per line.\n";	L25
4: return	return std::cout.flush() ? 0 : 1;	L28
5: branch	std::cout.flush()	L28
6: then	then	L28
7: statement	0	L28
6: else	else	L28
7: statement	1	L28
2: branch	(std::string_view(argv[1]) != "--" argc < 3)	L30
3: then	then	L30
4: statement	std::cerr << "Use --help for usage.\n";	L31

Depth / structure	Captured label	Source
4: return	return 2;	L32
2: statement	foundation::Store store;	L34
2: loop	for (int i = 2; i < argc; ++i)	L36
3: statement	store.add(argv[i]);	L36
2: loop	for (const auto& row : store.snapshot())	L37
3: statement	std::cout << row.id << '\t' << row.text << '\n';	L38
2: branch	(!std::cout.flush())	L40
3: then	then	L40
4: return	return 1;	L40
1: catch	catch (const std::exception& error)	L41
2: statement	std::cerr << error.what() << '\n';	L42
2: return	return 1;	L43
0: return	return 0;	L45

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::advance_input_epoch

method | [gui/shared/application.cpp:67](#) | Lines 67-72 | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui::Application

Captured signature

void Application::advance_input_epoch()

Why this implementation is on an edit path

[core-operation](#) / N3: 3. Resolve the selected ID (EDIT)

Why: A selected string is an identity, not a vector index. Returned snapshots are owning copies.

[commands](#) / N2: 2. Implement one command helper (NEW)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: A menu action and a button/shortcut can call the same helper.

[import-export](#) / N4: 4. Preserve bounds and freshness (REUSE)

Why: A format parser alone does not authorize an unmatched service result or stale input.

Complete captured source

[gui/shared/application.cpp](#) | Lines 67-72 | SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
67 void Application::advance_input_epoch() {
68     if (input_epoch_ == std::numeric_limits<std::uint64_t>::max()) {
69         throw std::overflow_error("Input epoch exhausted");
70     }
71     ++input_epoch_;
72 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 13 indexed candidate call sites across 6 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::append_entry (gui/shared/application.cpp:49)	L50	name-candidate
foundation::ui::Application::complete_service (gui/shared/application.cpp:232)	L236	name-candidate
foundation::ui::Application::complete_task (gui/shared/application.cpp:281)	L287	name-candidate
foundation::ui::Application::handle::anonymous lambda @93 (gui/shared/application.cpp:93)	L96, L106, L112, L117, L128, L146, L150, L159	name-candidate
foundation::ui::Application::enable_remove_feature (gui/shared/application.cpp:168)	L170	name-candidate
foundation::ui::Application::next_service (gui/shared/application.cpp:223)	L225	name-candidate

Outgoing: 2 detected call sites grouped into 2 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
std::numeric_limits<std::uint64_t>::max L68	No indexed target; external / unresolved.	unresolved
std::overflow_error L69	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

4 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(input_epoch_ == std::numeric_limits<std::uint64_t>::max())	L68
1: then	then	L68
2: throw	throw std::overflow_error("Input epoch exhausted");	L69

Depth / structure	Captured label	Source
0: statement	++input_epoch_;	L71

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::KeyBinding

`struct` | [@gui-boundary/include/gui/contract.hpp:119](#) | [Lines 119-123](#) | [Interactive symbol / full call lists](#)

Owner: `gui`

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/contract.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing `type_identifier`), line 330 (missing `type_identifier`), line 371 (missing `type_identifier`). Recovered structure may be incomplete.

Captured signature

```
struct KeyBinding
```

Why this implementation is on an edit path

[commands / N5: 5. Declare a supported key binding \(EDIT\)](#)

Why: A shortcut is declared in the shared Snapshot, independent of a native toolkit callback.

Complete captured source

[@gui-boundary/include/gui/contract.hpp](#) | [Lines 119-123](#) | [SHA-256 f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00ffd9db629a404d40da8](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
119 | struct KeyBinding {
120 |     ShortcutKey key=ShortcutKey::escape;
121 |     WidgetKey target;
122 |     bool control=false, shift=false, alt=false;
123 | };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

3 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	ShortcutKey key=ShortcutKey::escape;	L120
0: statement	WidgetKey target;	L121
0: statement	bool control=false,shift=false,alt=false;	L122

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::ShortcutKey

enum | [@gui-boundary/include/gui/contract.hpp:118](#) | Lines 118-118 | [Interactive symbol / full call lists](#)

Owner: gui

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

archive: [third_party/gui-inputs/gui-inputs.tar.gz](#) | member: [upstream/include/gui/contract.hpp](#) | revision: [7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing type_identifier), line 330 (missing type_identifier), line 371 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

enum class ShortcutKey

Why this implementation is on an edit path

[commands / N5: 5. Declare a supported key binding \(EDIT\)](#)

Why: A shortcut is declared in the shared Snapshot, independent of a native toolkit callback.

Complete captured source

[@gui-boundary/include/gui/contract.hpp](#) | Lines 118-118 | SHA-256 [f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00fd9db629a404d40da8](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
118 | enum class ShortcutKey { escape,enter,f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f12 };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

1 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	{ escape,enter,f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f12 }	L118

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

gui::normalize_event

[function](#) | [@gui-boundary/include/gui/contract.hpp:371](#) | [Lines 371-393](#) | [Interactive symbol / full call lists](#)

Owner: gui

Read-only supplier reference. This captured upstream definition is evidence for the supplier contract; follow the edit map for application changes.

[archive: third_party/gui-inputs/gui-inputs.tar.gz](#) | [member: upstream/include/gui/contract.hpp](#) | [revision: 7a704f73e563a167ea335dd23ccd9f383ebec274](#)

Scanner caveats for this file:

Syntax parser recovered from errors: line 189 (missing type_identifier), line 330 (missing type_identifier), line 371 (missing type_identifier). Recovered structure may be incomplete.

Captured signature

```
inline bool normalize_event(const Snapshot& view,Event& event,const ScrollLookup& scroll={})
```

Why this implementation is on an edit path

[commands / N5: 5. Declare a supported key binding \(EDIT\)](#)

Why: A shortcut is declared in the shared Snapshot, independent of a native toolkit callback.

[commands / N6: 6. Normalize and project state \(REUSE\)](#)

Why: Menu, button and shortcut behavior can converge on the same shared action, with normal stale/disabled input rejection.

Complete captured source

[@gui-boundary/include/gui/contract.hpp](#) | [Lines 371-393](#) | [SHA-256 f78b419b1a1e02ce808d85b3f7e8540dad7628c461a00ffd9db629a404d40da8](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
371 inline bool normalize_event(const Snapshot& view,Event& event,const ScrollLookup& scroll={}) {
372     if(const auto* key=std::get_if<ShortcutEvent>(&event)) {
373         std::optional<WidgetKey> target;
374         for(const auto& binding:view.key_bindings)
375             if(binding.key==key->key&&binding.control==key->control&&binding.shift==key->shift&&binding.alt==key->alt){targ
+ et=binding.target;break;}
376         if(!target)return false;
377         event=WidgetEvent{*target,Activate{}};
378     }
```

```
379     return std::visit([&](auto& value)->bool {
380         using T=std::decay_t<decltype(value)>;
381         if constexpr(std::is_same_v<T,WidgetEvent>)return normalize_widget_event(view,value,scroll);
382         else if constexpr(std::is_same_v<T,PageEvent>) {
383             if(view.modal_root)return false;
384             for(const auto& page:view.pages)if(page.id==value.id)
385                 return page.enabled&&page.visible&&view.active_page!=page.id;
386             return false;
387         } else if constexpr(std::is_same_v<T,ResizeEvent>) {
388             return valid_rect({0,0,value.client_size.width,value.client_size.height})&&
389                 std::isfinite(value.display_scale)&&value.display_scale>0&&value.display_scale<=16;
390         } else if constexpr(std::is_same_v<T,ShortcutEvent>)return false;
391         else return true;
392     },event);
393 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 2 indexed candidate call sites across 2 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
Example::handle (@gui-boundary/examples/application.hpp:67)	L76	name-candidate
foundation::ui::Application::handle (gui/shared/application.cpp:74)	L85	name-candidate

Outgoing: 2 detected call sites grouped into 2 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
std::get_if<ShortcutEvent> L372	No indexed target; external / unresolved.	unresolved
std::visit L379	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

13 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(const auto* key=std::get_if<ShortcutEvent>(&event))	L372
1: then	then	L372
2: statement	std::optional<WidgetKey> target;	L373
2: loop	for(const auto& binding:view.key_bindings)	L374

Depth / structure	Captured label	Source
3: branch	(binding.key==key->key&&binding.control==key->control&&binding.shift==key->shift&&binding.alt==key->alt)	L375
4: then	then	L375
5: statement	target=binding.target;	L375
5: break	break;	L375
2: branch	(!target)	L376
3: then	then	L376
4: return	return false;	L376
2: statement	event=WidgetEvent{*target,Activate{}};	L377
0: return	return std::visit([&](auto& value)->bool { using T=std::decay_t<decltype(value)>; if constexpr(std::is_same_v<T,WidgetEvent>)return normalize_widget_event(view,value,scroll); else...	L379

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::TaskUpdate

[struct](#) | [gui/shared/task.hpp:12](#) | [Lines 12-16](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui

Captured signature

struct TaskUpdate

Why this implementation is on an edit path

[background-task](#) / [N1: 1. Define the result/progress contract \(DECISION\)](#)

Why: The current completion validator encodes counting semantics; a different numeric result may legitimately exceed processed.

Complete captured source

[gui/shared/task.hpp](#) | [Lines 12-16](#) | [SHA-256 407e841598299b41c8a4e726a44341d05dbeb5e5b46b5b0098d8a2b09e10a9fe](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
12 | struct TaskUpdate {
13 |     std::uint64_t generation = 0;
14 |     std::size_t processed = 0, total = 0, result = 0;
15 |     bool complete = false;
16 | };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

3 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	std::uint64_t generation = 0;	L13
0: statement	std::size_t processed = 0, total = 0, result = 0;	L14
0: statement	bool complete = false;	L15

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::complete_task

[method](#) | [gui/shared/application.cpp:281](#) | [Lines 281-294](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui::Application

Captured signature

bool Application::complete_task(const TaskUpdate& update)

Why this implementation is on an edit path

[background-task / N1: 1. Define the result/progress contract \(DECISION\)](#)

Why: The current completion validator encodes counting semantics; a different numeric result may legitimately exceed processed.

[background-task / N5: 5. Interpret completion honestly \(EDIT\)](#)

Why: Current guards reject result > processed, and require complete to mean processed == total; keep or deliberately revise these with the contract.

Complete captured source

[gui/shared/application.cpp](#) | [Lines 281-294](#) | [SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
281 | bool Application::complete_task(const TaskUpdate& update) {
282 |     // Value-only completion identity survives cancellation and replacement.
283 |     if (adapter_.closed() || shutdown_ || !task_running_ || update.generation != task_progress_.generation ||
```

```
284         update.total != task_progress_.total || update.processed < task_progress_.processed ||
285         update.processed > update.total || update.result > update.processed ||
286         update.result < task_progress_.result || update.complete != (update.processed == update.total)) return false;
287     if (update.complete) advance_input_epoch();
288     task_progress_ = update;
289     task_running_ = !update.complete;
290     task_status_ = update.complete ? "Counted " + std::to_string(update.result) + " non-space bytes" :
291     "Processed " + std::to_string(update.processed) + " / " + std::to_string(update.total) + " bytes";
292     publish();
293     return true;
294 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 6 indexed candidate call sites across 2 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::tick (gui/shared/application.cpp:274)	L277	name-candidate
application_task (gui/tests/task_test.cpp:29)	L47, L53, L54, L66, L70	name-candidate

Outgoing: 6 detected call sites grouped into 4 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
advance_input_epoch L287	foundation::ui::Application::advance_input_epoch (gui/shared/application.cpp:67)	name-candidate
publish L292	foundation::ui::Application::publish (gui/shared/application.cpp:300) Example::publish (@gui-boundary/examples/application.hpp:150) anonymous namespace @5::App::publish (gui/tests/bezel_test.cpp:39)	ambiguous-name
adapter_.closed L283	No indexed target; external / unresolved.	unresolved
std::to_string L290, L291	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

16 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(adapter_.closed() shutdown_ !task_running_ update.generation != task_progress_.generation update.total != task_progress_.total update.processed < task_progress_.pr...	L283
1: then	then	L286

Depth / structure	Captured label	Source
2: return	return false;	L286
0: branch	(update.complete)	L287
1: then	then	L287
2: statement	advance_input_epoch();	L287
0: statement	task_progress_ = update;	L288
0: statement	task_running_ = !update.complete;	L289
0: statement	task_status_ = update.complete ? "Counted " + std::to_string(update.result) + " non-space bytes" : "Processed " + std::to_string(update.processed) + " / " + std::to_string(update....	L290
1: branch	update.complete	L290
2: then	then	L290
3: statement	"Counted " + std::to_string(update.result) + " non-space bytes"	L290
2: else	else	L291
3: statement	"Processed " + std::to_string(update.processed) + " / " + std::to_string(update.total) + " bytes"	L291
0: statement	publish();	L292
0: return	return true;	L293

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::TextTask::advance

method | [gui/shared/task.hpp:39](#) | [Lines 39-52](#) | [Interactive symbol / full call lists](#)

Owner: foundation::ui::TextTask

Captured signature

std::optional<TaskUpdate> advance(std::size_t budget = 256)

Why this implementation is on an edit path

[background-task / N2: 2. Change bounded computation \(EDIT\)](#)

Why: The existing result increments for non-space bytes; changing only the UI label does not change the computation.

Complete captured source

[gui/shared/task.hpp](#) | [Lines 39-52](#) | [SHA-256 407e841598299b41c8a4e726a44341d05dbeb5e5b46b5b0098d8a2b09e10a9fe](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
39     std::optional<TaskUpdate> advance(std::size_t budget = 256) {
40         if (!running_) return std::nullopt;
41         if (!budget || budget > 4096) throw std::invalid_argument("Task step budget must be 1..4096 bytes");
42         while (budget && row_ < input_.size()) {
43             const auto& text = input_[row_];
44             if (offset_ == text.size()) { ++row_; offset_ = 0; continue; }
45             update_.result += text[offset_++] != ' ';
46             ++update_.processed; --budget;
47         }
48         if (update_.processed == update_.total) {
49             update_.complete = true; running_ = false; input_.clear();
50         }
51         return update_;
52     }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 14 indexed candidate call sites across 7 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::tick (gui/shared/application.cpp:274)	L277	ambiguous-name
foundation::ui::CooperativeTaskExecutor::advance (gui/shared/task.hpp:70)	L70	ambiguous-name
foundation::ui::anonymous namespace @7::NativeTaskExecutor::run (gui/shared/task_native.cpp:20)	L28	ambiguous-name
anonymous namespace @5::SlowExecutor::advance (gui/tests/bezel_test.cpp:223)	L223	ambiguous-name
finish (gui/tests/task_native_test.cpp:8)	L11	ambiguous-name
main (gui/tests/task_native_test.cpp:21)	L26, L37	ambiguous-name
owned_task (gui/tests/task_test.cpp:13)	L17, L18, L19, L20, L22, L23, L27	ambiguous-name

Outgoing: 4 detected call sites grouped into 4 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
std::invalid_argument L41	No indexed target; external / unresolved.	unresolved
input_.size L42	foundation::Store::size (src/store.cpp:73)	name-candidate
text.size L44	foundation::Store::size (src/store.cpp:73)	name-candidate

Call name / call sites	Candidate target names / captured locations	Analysis
<code>input_.clear</code> L49	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

22 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(!running_)	L40
1: then	then	L40
2: return	return std::nullopt;	L40
0: branch	(!budget budget > 4096)	L41
1: then	then	L41
2: throw	throw std::invalid_argument("Task step budget must be 1..4096 bytes");	L41
0: loop	while (budget && row_ < input_.size())	L42
1: statement	const auto& text = input_[row_];	L43
1: branch	(offset_ == text.size())	L44
2: then	then	L44
3: statement	++row_;	L44
3: statement	offset_ = 0;	L44
3: continue	continue;	L44
1: statement	update_.result += text[offset_++] != ' ';	L45
1: statement	++update_.processed;	L46
1: statement	--budget;	L46
0: branch	(update_.processed == update_.total)	L48
1: then	then	L48
2: statement	update_.complete = true;	L49
2: statement	running_ = false;	L49
2: statement	input_.clear();	L49
0: return	return update_;	L51

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::TaskExecutor

class | [gui/shared/task.hpp:58](#) | [Lines 58-65](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui

Captured signature

class TaskExecutor

Why this implementation is on an edit path

[background-task](#) / [N4: 4. Reuse scheduling and tick delivery \(REUSE\)](#)

Why: Scheduling is selected once for the platform; task logic does not need a separate implementation in each GUI renderer.

Owning type: 1 indexed members in this declaration file. Members not recognized by the syntax scanner remain visible in the complete source below.

Indexed member	Kind / line	Signature
foundation::ui::TaskExecutor::~~TaskExecutor	method L60	virtual ~TaskExecutor() = default;

Complete captured source

[gui/shared/task.hpp](#) | [Lines 58-65](#) | [SHA-256 407e841598299b41c8a4e726a44341d05dbeb5e5b46b5b0098d8a2b09e10a9fe](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
58 | class TaskExecutor {
59 | public:
60 |     virtual ~TaskExecutor() = default;
61 |     virtual TaskUpdate start(std::vector<std::string>) = 0;
62 |     virtual std::optional<TaskUpdate> advance() = 0;
63 |     virtual void cancel() noexcept = 0;
64 |     virtual void shutdown() noexcept = 0;
65 | };
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 0 indexed candidate call sites across 0 unique caller symbols.

No indexed incoming name candidates.

Outgoing: 0 detected call sites grouped into 0 distinct name / analysis / target-set rows.

No calls detected in this captured symbol.

Complete indexed structural outline

5 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	public	L59
0: statement	virtual TaskUpdate start(std::vector<std::string>) = 0;	L61
0: statement	virtual std::optional<TaskUpdate> advance() = 0;	L62
0: statement	virtual void cancel() noexcept = 0;	L63
0: statement	virtual void shutdown() noexcept = 0;	L64

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::tick

[method](#) | [gui/shared/application.cpp:274](#) | [Lines 274-279](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui::Application

Captured signature

void Application::tick()

Why this implementation is on an edit path

[background-task](#) / [N4: 4. Reuse scheduling and tick delivery \(REUSE\)](#)

Why: Scheduling is selected once for the platform; task logic does not need a separate implementation in each GUI renderer.

Complete captured source

[gui/shared/application.cpp](#) | [Lines 274-279](#) | [SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
274 void Application::tick() {
275     if (adapter_.closed()) { shutdown(); return; }
276     if (shutdown_) return;
277     if (const auto update = task_->advance()) complete_task(*update);
278     retry_presentation();
279 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 27 indexed candidate call sites across 23 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::host::Browser::Browser::anonymous lambda @18 (gui/host/browser.hpp:18)	L18	ambiguous-name
foundation::host (gui/host/contract.hpp:7)	L14	ambiguous-name
foundation::host::Session::tick (gui/host/contract.hpp:62)	L63	ambiguous-name
foundation::host::FramebufferHost::prepare_bezel (gui/host/framebuffer.hpp:50)	L52	ambiguous-name
foundation::host::FramebufferHost::bezel_button::anonymous lambda @73 (gui/host/framebuffer.hpp:73)	L75	ambiguous-name
foundation::host::FramebufferHost::resize (gui/host/framebuffer.hpp:88)	L89	ambiguous-name
foundation::host::FramebufferHost::contact (gui/host/framebuffer.hpp:91)	L93	ambiguous-name
foundation::host::FramebufferHost::cancel (gui/host/framebuffer.hpp:96)	L96	ambiguous-name

Print scope: 15 additional caller symbols containing 19 candidate call sites are omitted. Map-linked and same-file callers are listed first. [All callers in explorer](#)

Outgoing: 5 detected call sites grouped into 5 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
shutdown L275	foundation::ui::Application::shutdown (gui/shared/application.cpp:266) foundation::host::AdapterServices::shutdown (gui/host/contract.hpp:34) foundation::host::FileServices::shutdown (gui/host/file_services.hpp:164) +6 additional candidate targets omitted; all targets	ambiguous-name
complete_task L277	foundation::ui::Application::complete_task (gui/shared/application.cpp:281)	name-candidate
task_>advance L277	foundation::ui::TextTask::advance (gui/shared/task.hpp:39) foundation::ui::CooperativeTaskExecutor::advance (gui/shared/task.hpp:70) foundation::ui::anonymous namespace @7::NativeTaskExecutor::advance (gui/shared/task_native.cpp:51) +1 additional candidate targets omitted; all targets	ambiguous-name
retry_presentation L278	foundation::ui::Application::retry_presentation (gui/shared/application.cpp:296) Example::retry_presentation (@gui-boundary/examples/application.hpp:61) anonymous namespace @5::App::retry_presentation (gui/tests/bezel_test.cpp:62) +1 additional candidate targets omitted; all targets	ambiguous-name
adapter_.closed L275	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

11 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(adapter_.closed())	L275

Depth / structure	Captured label	Source
1: then	then	L275
2: statement	shutdown();	L275
2: return	return;	L275
0: branch	(shutdown_)	L276
1: then	then	L276
2: return	return;	L276
0: branch	(const auto update = task_->advance())	L277
1: then	then	L277
2: statement	complete_task(*update);	L277
0: statement	retry_presentation();	L278

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::shutdown

method | [gui/shared/application.cpp:266](#) | Lines 266-272 | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::ui::Application

Captured signature

void Application::shutdown() noexcept

Why this implementation is on an edit path

[background-task](#) / N6: 6. Project controls and close owned work (REUSE)

Why: Responsiveness includes cancellation, late update rejection and complete owned-work cleanup, not just showing progress.

Complete captured source

[gui/shared/application.cpp](#) | Lines 266-272 | SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
266 | void Application::shutdown() noexcept {
267 |     if (shutdown_) return;
268 |     if (input_epoch_ != std::numeric_limits<std::uint64_t>::max()) ++input_epoch_;
269 |     shutdown_ = true;
270 |     pending_services_ = 0;
271 |     task_->shutdown(); task_running_ = false; services_.shutdown();
272 | }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 20 indexed candidate call sites across 17 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::ui::Application::handle (gui/shared/application.cpp:74)	L78	ambiguous-name
foundation::ui::Application::shutdown (gui/shared/application.cpp:266)	L271	ambiguous-name
foundation::ui::Application::tick (gui/shared/application.cpp:274)	L275	ambiguous-name
Example::handle (@gui-boundary/examples/application.hpp:67)	L71	ambiguous-name
foundation::host::Browser::~~Browser (gui/host/browser.hpp:21)	L21	ambiguous-name
foundation::host (gui/host/contract.hpp:7)	L15	ambiguous-name
foundation::host::Session::~~Session (gui/host/contract.hpp:48)	L48	ambiguous-name
foundation::host::Session::services (gui/host/contract.hpp:49)	L50	ambiguous-name

Print scope: 9 additional caller symbols containing 10 candidate call sites are omitted. Map-linked and same-file callers are listed first. [All callers in explorer](#)

Outgoing: 3 detected call sites grouped into 3 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
task_>shutdown L271	foundation::ui::Application::shutdown (gui/shared/application.cpp:266) foundation::host::AdapterServices::shutdown (gui/host/contract.hpp:34) foundation::host::FileServices::shutdown (gui/host/file_services.hpp:164) +6 additional candidate targets omitted; all targets	ambiguous-name
services_.shutdown L271	foundation::ui::Application::shutdown (gui/shared/application.cpp:266) foundation::host::AdapterServices::shutdown (gui/host/contract.hpp:34) foundation::host::FileServices::shutdown (gui/host/file_services.hpp:164) +6 additional candidate targets omitted; all targets	ambiguous-name
std::numeric_limits<std::uint64_t>::max L268	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

11 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(shutdown_)	L267
1: then	then	L267
2: return	return;	L267

Depth / structure	Captured label	Source
0: branch	(input_epoch_ != std::numeric_limits<std::uint64_t>::max())	L268
1: then	then	L268
2: statement	++input_epoch_;	L268
0: statement	shutdown_ = true;	L269
0: statement	pending_services_ = 0;	L270
0: statement	task_->shutdown();	L271
0: statement	task_running_ = false;	L271
0: statement	services_.shutdown();	L271

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::ui::Application::complete_service

method | [gui/shared/application.cpp:232](#) | Lines 232-264 | [Interactive symbol / full call lists](#)

Owner: foundation::ui::Application

Captured signature

```
bool Application::complete_service(gui::ServiceResult result)
```

Why this implementation is on an edit path

[import-export / N1: 1. Separate content from transport \(DECISION\)](#)

Why: Changing a content format does not require copying application meaning into every backend.

[import-export / N3: 3. Parse into a replacement Store \(EDIT\)](#)

Why: Mutating the live Store while parsing would expose a partial import on rejection.

[import-export / N4: 4. Preserve bounds and freshness \(REUSE\)](#)

Why: A format parser alone does not authorize an unmatched service result or stale input.

[import-export / N6: 6. Project honest completion state \(EDIT\)](#)

Why: The visible result must distinguish a successfully parsed replacement, rejected content and host handoff.

Complete captured source

[gui/shared/application.cpp](#) | Lines 232-264 | SHA-256 d6e39c21d8e62ec228f6c3f2f10eeb85f4c50c77833bcf24e1a6d5f012e5828d

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
232 | bool Application::complete_service(gui::ServiceResult result) {
233 |     if (shutdown_ || adapter_.closed()) return false;
234 |     const auto request = services_.current();
```

```
235     if (!request || request->id != result.id) return false;
236     advance_input_epoch();
237     if (!services_.complete(result)) return false;
238     status_error_ = result.status == gui::ServiceStatus::error;
239     if (status_error_) status_ = std::move(result.error);
240     else if (result.status == gui::ServiceStatus::cancelled) status_ = "Service cancelled";
241     else if (request->kind == gui::ServiceKind::prompt) get("entries.heading").state.text = std::move(result.value);
242     else if (request->kind == gui::ServiceKind::write_text) status_ = "Export handed to host";
243     else if (request->kind == gui::ServiceKind::read_text) {
244         try {
245             auto replacement = entries_; // Commit all records together; preserve ID monotonicity.
246             for (const auto& record : replacement.snapshot()) replacement.erase(record.id);
247             std::size_t begin = 0;
248             while (begin < result.value.size()) {
249                 const auto end = result.value.find('\n', begin);
250                 auto line = std::string_view(result.value).substr(begin,
251                     end == std::string::npos ? end : end - begin);
252                 if (!line.empty() && line.back() == '\r') line.remove_suffix(1);
253                 replacement.add(line);
254                 if (end == std::string::npos) break;
255                 begin = end + 1;
256             }
257             entries_ = std::move(replacement);
258             get("entries.list").state.selected.reset();
259             status_ = "Entries imported";
260         } catch (const std::exception& error) { status_ = error.what(); status_error_ = true; }
261     }
262     publish();
263     return true;
264 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 9 indexed candidate call sites across 7 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::host::Browser::Browser::anonymous lambda @17 (gui/host/browser.hpp:17)	L17	ambiguous-name
foundation::host (gui/host/contract.hpp:7)	L18	ambiguous-name
foundation::host::Session::services (gui/host/contract.hpp:49)	L51	ambiguous-name
foundation::host::Session::services::anonymous lambda @58 (gui/host/contract.hpp:58)	L59	ambiguous-name

Caller / captured location	Call-site lines	Analysis
anonymous namespace @9::scenario (gui/tests/application_test.cpp:40)	L110, L112, L115	ambiguous-name
anonymous namespace @9::shutdown_without_close (gui/tests/application_test.cpp:219)	L229	ambiguous-name
main (gui/tests/file_services_test.cpp:20)	L80	ambiguous-name

Outgoing: 22 detected call sites grouped into 19 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
advance_input_epoch L236	foundation::ui::Application::advance_input_epoch (gui/shared/application.cpp:67)	name-candidate
replacement.add L253	foundation::ui::Application::add (gui/shared/application.cpp:29) foundation::Store::add (src/store.cpp:33) Example::add (@gui-boundary/examples/application.hpp:132) +3 additional candidate targets omitted; all targets	ambiguous-name
publish L262	foundation::ui::Application::publish (gui/shared/application.cpp:300) Example::publish (@gui-boundary/examples/application.hpp:150) anonymous namespace @5::App::publish (gui/tests/bezel_test.cpp:39)	ambiguous-name
adapter_.closed L233	No indexed target; external / unresolved.	unresolved
services_.current L234	No indexed target; external / unresolved.	unresolved
services_.complete L237	No indexed target; external / unresolved.	unresolved
std::move L239, L241, L257	No indexed target; external / unresolved.	unresolved
get L241, L258	Example::get (@gui-boundary/examples/application.hpp:141) foundation::ui::Application::get (gui/shared/application.hpp:58) foundation::Store::get (src/store.cpp:65)	ambiguous-name

Print scope: 11 additional outgoing groups containing 11 call sites are omitted. Groups containing map-linked or same-file candidates are listed first. [All outgoing calls in explorer](#)

Complete indexed structural outline

60 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: branch	(shutdown_ adapter_.closed())	L233
1: then	then	L233
2: return	return false;	L233
0: statement	const auto request = services_.current();	L234

Depth / structure	Captured label	Source
0: branch	(!request request->id != result.id)	L235
1: then	then	L235
2: return	return false;	L235
0: statement	advance_input_epoch();	L236
0: branch	(!services_.complete(result))	L237
1: then	then	L237
2: return	return false;	L237
0: statement	status_error_ = result.status == gui::ServiceStatus::error;	L238
0: branch	(status_error_)	L239
1: then	then	L239
2: statement	status_ = std::move(result.error);	L239
1: else	else	L240
2: branch	(result.status == gui::ServiceStatus::cancelled)	L240
3: then	then	L240
4: statement	status_ = "Service cancelled";	L240
3: else	else	L241
4: branch	(request->kind == gui::ServiceKind::prompt)	L241
5: then	then	L241
6: statement	get("entries.heading").state.text = std::move(result.value);	L241
5: else	else	L242
6: branch	(request->kind == gui::ServiceKind::write_text)	L242
7: then	then	L242
8: statement	status_ = "Export handed to host";	L242
7: else	else	L243
8: branch	(request->kind == gui::ServiceKind::read_text)	L243
9: then	then	L243
10: try	try	L244
11: try_body	try body	L244

Depth / structure	Captured label	Source
12: statement	auto replacement = entries_;	L245
12: loop	for (const auto& record : replacement.snapshot())	L246
13: statement	replacement.erase(record.id);	L246
12: statement	std::size_t begin = 0;	L247
12: loop	while (begin < result.value.size())	L248
13: statement	const auto end = result.value.find("\n", begin);	L249
13: statement	auto line = std::string_view(result.value).substr(begin, end == std::string::npos ? end : end - begin);	L250
14: branch	end == std::string::npos	L251
15: then	then	L251
16: statement	end	L251
15: else	else	L251
16: statement	end - begin	L251
13: branch	(!line.empty() && line.back() == '\r')	L252
14: then	then	L252
15: statement	line.remove_suffix(1);	L252
13: statement	replacement.add(line);	L253
13: branch	(end == std::string::npos)	L254
14: then	then	L254
15: break	break;	L254
13: statement	begin = end + 1;	L255
12: statement	entries_ = std::move(replacement);	L257
12: statement	get("entries.list").state.selected.reset();	L258
12: statement	status_ = "Entries imported";	L259
11: catch	catch (const std::exception& error)	L260
12: statement	status_ = error.what();	L260
12: statement	status_error_ = true;	L260
0: statement	publish();	L262
0: return	return true;	L263

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::host::file_detail::transfer

[function](#) | [gui/host/file_services.hpp:66](#) | [Lines 66-119](#) | [Interactive symbol](#) / [full call lists](#)

Owner: foundation::host::file_detail

Captured signature

```
inline gui::ServiceResult transfer(const gui::ServiceRequest& request, const std::filesystem::path& path, const std::atomic_bool& stop)
```

Why this implementation is on an edit path

[import-export](#) / [N5: 5. Reuse native/browser byte transport \(REUSE\)](#)

Why: Native and browser hosts own their file-selection/transfer authority while the application owns content meaning.

Complete captured source

[gui/host/file_services.hpp](#) | [Lines 66-119](#) | [SHA-256 65b0c9a2d4da5f2d4823b47c4f9448d99fa5d0ff23320cb7c5099353827678af](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
66 inline gui::ServiceResult transfer(const gui::ServiceRequest& request,
67                                   const std::filesystem::path& path, const std::atomic_bool& stop) {
68     gui::ServiceResult result{request.id, gui::ServiceStatus::success, {}, {}};
69     bool committed = false;
70     if (request.byte_limit > 65536 || request.value.size() > request.byte_limit)
71         throw std::length_error("File content exceeds its byte limit");
72     if (request.kind == gui::ServiceKind::read_text) {
73         File file(path, false);
74         if (file.fd < 0 || !file.regular()) throw std::runtime_error("Select a readable regular file");
75         char buffer[4096];
76         while (!stop) {
77             const int count = file.read(buffer, sizeof buffer);
78             if (count < 0) { if (errno == EINTR) continue; throw std::runtime_error("Cannot read selected file"); }
79             if (count == 0) break;
80             if (static_cast<std::size_t>(count) > request.byte_limit - result.value.size())
81                 throw std::length_error("Selected file exceeds the byte limit");
82             result.value.append(buffer, static_cast<std::size_t>(count));
83         }
84     } else if (request.kind == gui::ServiceKind::write_text) {
85         // Exclusive temporary creation avoids truncating the chosen destination
86         // on cancellation or a failed write. Rename replaces it only on success.
87         static std::atomic_uint64_t serial{0};
88         auto temporary = path;
89         temporary += ".foundation-" + std::to_string(std::chrono::steady_clock::now().time_since_epoch().count()) +
```

```
90         "-" + std::to_string(++serial) + ".tmp";
91     File file(temporary, true);
92     if (file.fd < 0) throw std::runtime_error("Cannot create export beside selected destination");
93     struct Cleanup {
94         File& file; std::filesystem::path path;
95         ~Cleanup() { file.close(); std::error_code error; std::filesystem::remove(path, error); }
96     } cleanup{file, temporary};
97     std::size_t offset = 0;
98     while (offset < request.value.size() && !stop) {
99         const int count = file.write(request.value.data() + offset,
100             static_cast<unsigned>(std::min<std::size_t>(4096, request.value.size() - offset)));
101         if (count < 0 && errno == EINTR) continue;
102         if (count <= 0) throw std::runtime_error("Cannot write selected file");
103         offset += static_cast<std::size_t>(count);
104     }
105     if (!file.close()) throw std::runtime_error("Cannot close exported file");
106     if (!stop) {
107 #ifdef _WIN32
108         if (!MoveFileExW(temporary.c_str(), path.c_str(), MOVEFILE_REPLACE_EXISTING))
109             throw std::runtime_error("Cannot replace selected destination");
110 #else
111         std::error_code error; std::filesystem::rename(temporary, path, error);
112         if (error) throw std::runtime_error("Cannot replace selected destination");
113 #endif
114         committed = true;
115     }
116 } else throw std::invalid_argument("Unknown content service");
117 if (stop && !committed) { result.status = gui::ServiceStatus::cancelled; result.value.clear(); }
118 return result;
119 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 4 indexed candidate call sites across 3 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::host::FileServices::start::anonymous lambda @151 (gui/host/file_services.hpp:151)	L153	name-candidate
main::receive (gui/hosts/web_main.cpp:48)	L58	name-candidate
main (gui/tests/file_services_test.cpp:20)	L72, L76	name-candidate

Outgoing: 35 detected call sites grouped into 23 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
file.regular L74	foundation::host::file_detail::File::regular (gui/host/file_services.hpp:43)	name-candidate
file.read L77	foundation::host::file_detail::File::read (gui/host/file_services.hpp:51)	name-candidate
file.write L99	foundation::host::file_detail::File::write (gui/host/file_services.hpp:58)	name-candidate
file.close L105	foundation::host::file_detail::File::close (gui/host/file_services.hpp:34) anonymous namespace @18::Context::close (tools/windows_gl_probe.cpp:143)	ambiguous-name
request.value.size L70, L98, L100	foundation::Store::size (src/store.cpp:73)	name-candidate
std::length_error L71, L81	No indexed target; external / unresolved.	unresolved
std::runtime_error L74, L78, L92, L102, L105, L109, L112	No indexed target; external / unresolved.	unresolved
static_cast<std::size_t> L80, L82, L103	No indexed target; external / unresolved.	unresolved

Print scope: 15 additional outgoing groups containing 16 call sites are omitted. Groups containing map-linked or same-file candidates are listed first. [All outgoing calls in explorer](#)

Complete indexed structural outline

72 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	gui::ServiceResult result{request.id, gui::ServiceStatus::success , {}, {}};	L68
0: statement	bool committed = false;	L69
0: branch	(request.byte_limit > 65536 request.value.size() > request.byte_limit)	L70
1: then	then	L71
2: throw	throw std::length_error ("File content exceeds its byte limit");	L71
0: branch	(request.kind == gui::ServiceKind::read_text)	L72
1: then	then	L72
2: statement	File file(path, false);	L73
2: branch	(file.fd < 0 !file.regular())	L74
3: then	then	L74
4: throw	throw std::runtime_error ("Select a readable regular file");	L74

Depth / structure	Captured label	Source
2: statement	char buffer[4096];	L75
2: loop	while (!stop)	L76
3: statement	const int count = file.read(buffer, sizeof buffer);	L77
3: branch	(count < 0)	L78
4: then	then	L78
5: branch	(errno == EINTR)	L78
6: then	then	L78
7: continue	continue;	L78
5: throw	throw std::runtime_error("Cannot read selected file");	L78
3: branch	(count == 0)	L79
4: then	then	L79
5: break	break;	L79
3: branch	(static_cast<std::size_t>(count) > request.byte_limit - result.value.size())	L80
4: then	then	L81
5: throw	throw std::length_error("Selected file exceeds the byte limit");	L81
3: statement	result.value.append(buffer, static_cast<std::size_t>(count));	L82
1: else	else	L84
2: branch	(request.kind == gui::ServiceKind::write_text)	L84
3: then	then	L84
4: statement	static std::atomic_uint64_t serial{0};	L87
4: statement	auto temporary = path;	L88
4: statement	temporary += ".foundation-" + std::to_string(std::chrono::steady_clock::now().time_since_epoch().count()) + "-" + std::to_string(++serial) + ".tmp";	L89
4: statement	File file(temporary, true);	L91
4: branch	(file.fd < 0)	L92
5: then	then	L92
6: throw	throw std::runtime_error("Cannot create export beside selected destination");	L92
4: statement	struct Cleanup { File& file; std::filesystem::path path; ~Cleanup() { file.close(); std::error_code error; std::filesystem::remove(path, error); } } cleanup{file, temporary};	L93

Depth / structure	Captured label	Source
4: statement	std::size_t offset = 0;	L97
4: loop	while (offset < request.value.size() && !stop)	L98
5: statement	const int count = file.write(request.value.data() + offset, static_cast<unsigned>(std::min<std::size_t>(4096, request.value.size() - offset)));	L99
5: branch	(count < 0 && errno == EINTR)	L101
6: then	then	L101
7: continue	continue;	L101
5: branch	(count <= 0)	L102
6: then	then	L102
7: throw	throw std::runtime_error("Cannot write selected file");	L102
5: statement	offset += static_cast<std::size_t>(count);	L103
4: branch	(!file.close())	L105
5: then	then	L105
6: throw	throw std::runtime_error("Cannot close exported file");	L105
4: branch	(!stop)	L106
5: then	then	L106
6: preprocessor	#ifdef _WIN32	L107
7: statement	_WIN32	L107
7: branch	(!MoveFileExW(temporary.c_str(), path.c_str(), MOVEFILE_REPLACE_EXISTING))	L108
8: then	then	L109
9: throw	throw std::runtime_error("Cannot replace selected destination");	L109
7: preprocessor	#else	L110
8: statement	std::error_code error;	L111
8: statement	std::filesystem::rename(temporary, path, error);	L111
8: branch	(error)	L112
9: then	then	L112
10: throw	throw std::runtime_error("Cannot replace selected destination");	L112
6: statement	committed = true;	L114
3: else	else	L116

Depth / structure	Captured label	Source
4: throw	throw std::invalid_argument("Unknown content service");	L116
0: branch	(stop && !committed)	L117
1: then	then	L117
2: statement	result.status = gui::ServiceStatus::cancelled;	L117
2: statement	result.value.clear();	L117
0: return	return result;	L118

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

readTextFile

function | [gui/host/file_services.mjs:20](#) | [Lines 20-30](#) | [Interactive symbol](#) / [full call lists](#)

Entry-point clue: Exported declaration; module API entry candidate.

Captured signature

async function readTextFile(file, requestedLimit, {signal, isCurrent} = {})

Why this implementation is on an edit path

[import-export](#) / [N5: 5. Reuse native/browser byte transport \(REUSE\)](#)

Why: Native and browser hosts own their file-selection/transfer authority while the application owns content meaning.

Complete captured source

[gui/host/file_services.mjs](#) | [Lines 20-30](#) | [SHA-256 f5df8b2b19520cb6ee13f14b2ddc61cfa03691a306356cc2debdd58266f6d075](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
20 export async function readTextFile(file, requestedLimit, {signal, isCurrent} = {}) {
21   checkCurrent(signal, isCurrent);
22   const limit = byteLimit(requestedLimit);
23   if (!file || !Number.isSafeInteger(file.size) || file.size < 0 || file.size > limit)
24     throw new Error(`Select a file no larger than ${limit} bytes`);
25   // Slice and recheck protect against a provider returning more than advertised.
26   const bytes = await readSlice(file, 0, limit + 1, signal, isCurrent);
27   checkCurrent(signal, isCurrent);
28   if (bytes.byteLength > limit) throw new Error('Selected file exceeds the byte limit');
29   return new TextDecoder('utf-8', {fatal: true, ignoreBOM: true}).decode(bytes);
30 }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 7 indexed candidate call sites across 3 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
<code>executeFileService.anonymous lambda @98.anonymous lambda @127</code> <code>(gui/host/file_services.mjs:127)</code>	L136	name-candidate
<code><module> (gui/tests/browser_services_test.mjs:1)</code>	L129	name-candidate
<code><module> (gui/tests/file_services_test.mjs:1)</code>	L6, L8, L9, L10, L11	name-candidate

Outgoing: 9 detected call sites grouped into 7 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
<code>checkCurrent</code> L21, L27	<code>checkCurrent (gui/host/file_services.mjs:9)</code>	name-candidate
<code>byteLimit</code> L22	<code>byteLimit (gui/host/file_services.mjs:4)</code>	name-candidate
<code>readSlice</code> L26	<code>readSlice (gui/host/file_services.mjs:12)</code>	name-candidate
<code>Number.isSafeInteger</code> L23	No indexed target; external / unresolved.	unresolved
<code>Error</code> L24, L28	No indexed target; external / unresolved.	unresolved
<code>new TextDecoder('utf-8', {fatal: true, ignoreBOM: true}).decode</code> L29	<code>createWasmTransport.decode (gui/host/wasm_transport.mjs:24)</code>	name-candidate
<code>TextDecoder</code> L29	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

11 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	<code>checkCurrent(signal, isCurrent);</code>	L21
0: statement	<code>const limit = byteLimit(requestedLimit);</code>	L22
0: branch	<code>(!file !Number.isSafeInteger(file.size) file.size < 0 file.size > limit)</code>	L23
1: then	<code>then</code>	L24

Depth / structure	Captured label	Source
2: throw	throw new Error(`Select a file no larger than \${limit} bytes`);	L24
0: statement	const bytes = await readSlice(file, 0, limit + 1, signal, isCurrent);	L26
0: statement	checkCurrent(signal, isCurrent);	L27
0: branch	(bytes.byteLength > limit)	L28
1: then	then	L28
2: throw	throw new Error("Selected file exceeds the byte limit");	L28
0: return	return new TextDecoder('utf-8', {fatal: true, ignoreBOM: true}).decode(bytes);	L29

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation::detail::validate_text

function | [src/text_validation.cpp:32](#) | [Lines 32-34](#) | [Interactive symbol / full call lists](#)

Owner: foundation::detail

Scanner caveats for this file:

Syntax parser recovered from errors: line 6 (ERROR), line 9 (ERROR). Recovered structure may be incomplete.

Captured signature

uint32_t validate_text(const uint8_t *bytes, size_t length) noexcept

Why this implementation is on an edit path

[widget-runtime / N6: Selected text validator \(REUSE\)](#)

Why: Follow these deeper links when changing validation itself.

Complete captured source

[src/text_validation.cpp](#) | [Lines 32-34](#) | [SHA-256 509ec58625f832343e9e762e0b8dbf1df2519c67f2bf52839a1a9b990b2507bb](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
32 | uint32_t validate_text(const uint8_t *bytes, size_t length) noexcept {
33 |     return foundation_text_validate_v1(bytes, length);
34 | }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 2 indexed candidate call sites across 2 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
foundation::Store::validate (src/store.cpp:17)	L19	ambiguous-name
main (tests/rust_component_test.cpp:14)	L24	ambiguous-name

Outgoing: 1 detected call sites grouped into 1 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
foundation_text_validate_v1 L33	foundation_text_validate_v1 (src/text_validation.cpp:14)	name-candidate

Complete indexed structural outline

1 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: return	return foundation_text_validate_v1(bytes, length);	L33

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

foundation_text_validate_v1

[function](#) | [rust/text_validation/src/lib.rs:28](#) | [Lines 28-39](#) | [Interactive symbol](#) / [full call lists](#)

Entry-point clue: Declared extern function; foreign-call entry candidate.

Captured signature

pub unsafe extern "C" fn foundation_text_validate_v1(bytes: *const u8, length: usize) -> u32

Why this implementation is on an edit path

[widget-runtime / N6: Selected text validator \(REUSE\)](#)

Why: Follow these deeper links when changing validation itself.

Complete captured source

[rust/text_validation/src/lib.rs](#) | [Lines 28-39](#) | [SHA-256 40e5e80c7fde0da00914099ce161ab3c1da1ae80c42d290c9f7cdf397373693d](#)

Line numbers link to the offline source explorer. A '+' marks a wrapped continuation of the same logical line. Tabs are expanded to four spaces; non-ASCII characters are shown as escapes for portable PDF fonts.

```
28 | pub unsafe extern "C" fn foundation_text_validate_v1(bytes: *const u8, length: usize) -> u32 {
29 |     if length == 0 || length > MAX_TEXT_BYTES {
30 |         return INVALID_LENGTH;
31 |     }
32 |     if bytes.is_null() {
33 |         return INVALID_POINTER;
34 |     }
35 |     // The length is bounded and nonzero; the caller supplies the live allocation
```

```
36 | // and prevents mutation until return. The borrowed slice stays within this call.
37 | let text = unsafe { core::slice::from_raw_parts(bytes, length) };
38 | validate_text(text)
39 | }
```

Named call neighborhood

These are syntax-derived name candidates, not type-checked runtime edges. A dotted/member call can match unrelated types; unresolved calls may be external, dynamic, macros, or outside the snapshot. Empty caller lists do not rule out callbacks.

Incoming: 5 indexed candidate call sites across 1 unique caller symbols.

Caller / captured location	Call-site lines	Analysis
main (tests/rust_component_test.cpp:14)	L29, L34, L41, L48, L57	ambiguous-name

Outgoing: 3 detected call sites grouped into 3 distinct name / analysis / target-set rows.

Call name / call sites	Candidate target names / captured locations	Analysis
validate_text L38	validate_text (rust/text_validation/src/lib.rs:11)	name-candidate
bytes.is_null L32	No indexed target; external / unresolved.	unresolved
core::slice::from_raw_parts L37	No indexed target; external / unresolved.	unresolved

Complete indexed structural outline

12 captured landmarks. Read top to bottom; indentation follows lexical source nesting. Branches, loops, calls and exits are syntax landmarks, not an exact execution control-flow graph.

Depth / structure	Captured label	Source
0: statement	if length == 0 length > MAX_TEXT_BYTES { return INVALID_LENGTH; }	L29
1: branch	length == 0 length > MAX_TEXT_BYTES	L29
2: then	then	L29
3: statement	return INVALID_LENGTH;	L30
4: return	return INVALID_LENGTH	L30
0: statement	if bytes.is_null() { return INVALID_POINTER; }	L32
1: branch	bytes.is_null()	L32
2: then	then	L32
3: statement	return INVALID_POINTER;	L33
4: return	return INVALID_POINTER	L33

Depth / structure	Captured label	Source
0: statement	let text = unsafe { core::slice::from_raw_parts(bytes, length) };	L37
0: statement	validate_text(text)	L38

[Up to walkthrough index](#) | [Up to edit paths](#) | [Explore this symbol](#)

Source-only anchor: gui/shared/view_definition.hpp:22

Why this implementation is on an edit path

[widget / N1: 1. Declare the widget \(EDIT\)](#)

Why: This ordered table already supplies widget creation, ordering, and shared layout. A new rendering capability or new event type would be a separate contract change.

[widget-placement / N2: Set vertical order and height \(EDIT\)](#)

Why: A row in this table is the ordinary vertical placement edit; coordinates are computed later.

[tabs / N2: 2. Declare page-owned content \(EDIT\)](#)

Why: The current row schema has no page or parent field; an ID prefix cannot select a tab.

[core-operation / N2: 2. Add an Update action \(NEW\)](#)

Existing pattern for a planned addition. The proposed code is not a definition in this listing.

Why: Widget identity and command meaning belong to shared Application. Existing application.cpp is already compiled.

[commands / N4: 4. Route a current button \(EDIT\)](#)

Why: The retained contract requires every shortcut to target a current button, not a menu option ID.

[widget-runtime / N1: entries.add button \(REUSE\)](#)

Why: Use the same ID in declaration and dispatch.

[widget-placement / N1: Where should it appear? \(DECISION\)](#)

Why: The placement choice determines whether a row edit is enough.

Captured context excerpt

Bounded source context: lines 14-32 of 32 for anchors L18, L19, L22. 13 preceding and 0 following file lines are outside this excerpt. No containing callable/type was indexed for these anchors.

SHA-256 a2753be85be375a526aafaaa7fb6e9e917215eb3e760f9b1d7c8318ab981f20e | Snapshot 2026-10-04T23:16:35-05:00

```
14     bool bold = false;
15     gui::TextWrap wrap = gui::TextWrap::none;
16     bool remove_extension = false;
17 };
18 inline constexpr std::array view_definition{
19     ViewDefinition{"entries.form", gui::Kind::group, 0, "", "", ""},
20     ViewDefinition{"entries.heading", gui::Kind::label, 32, "Entry list", "", "", true},
21     ViewDefinition{"entries.editor", gui::Kind::text, 32, "", "New entry", "Type an entry"},
22     ViewDefinition{"entries.add", gui::Kind::button, 32, "", "Add entry", ""},
23     ViewDefinition{"entries.options", gui::Kind::menu, 32, "", "Actions", ""},
24     ViewDefinition{"entries.task.start", gui::Kind::button, 32, "", "Count text", ""},
25     ViewDefinition{"entries.task.cancel", gui::Kind::button, 32, "", "Cancel task", ""},
26     ViewDefinition{"entries.task.status", gui::Kind::label, 32, "Task ready", "", ""},
27     ViewDefinition{"entries.list", gui::Kind::list, 72, "", "Entries", "No entries"},
28     ViewDefinition{"entries.count", gui::Kind::label, 32, "", "", ""},
29     ViewDefinition{"entries.status", gui::Kind::label, 32, "", "", "", false, gui::TextWrap::word},
30     ViewDefinition{"entries.remove", gui::Kind::button, 32, "", "Remove selected", "", false, gui::TextWrap::none, true}
31 };
32 } // namespace foundation::ui
```

[Complete source in explorer](#) | [Up to walkthrough index](#) | [Up to edit paths](#)

Source-only anchor: rust/text_validation/src/lib.rs:1

Why this implementation is on an edit path

[source-files / N6: Rust module .rs \(EDIT\)](#)

Why: A new crate, dependency, or build.rs is a separate build-policy change, not this internal-module recipe.

Captured context excerpt

Bounded source context: lines 1-13 of 133 for anchors L1. 0 preceding and 120 following file lines are outside this excerpt. No containing callable/type was indexed for these anchors.

SHA-256 40e5e80c7fde0da00914099ce161ab3c1da1ae80c42d290c9f7cdf397373693d | Snapshot 2026-10-04T23:16:35-05:00

```
1  |[cfg_attr(not(test), no_std)]
2  |[deny(unsafe_op_in_unsafe_fn)]
3
4  const MAX_TEXT_BYTES: usize = 256;
5  const OK: u32 = 0;
6  const INVALID_LENGTH: u32 = 1;
7  const INVALID_ASCII: u32 = 2;
8  const INVALID_POINTER: u32 = 3;
9  const PROVIDER_RUST: u32 = 1;
10
11 fn validate_text(bytes: &[u8]) -> u32 {
12     if bytes.is_empty() || bytes.len() > MAX_TEXT_BYTES {
13         return INVALID_LENGTH;
```

[Complete source in explorer](#) | [Up to walkthrough index](#) | [Up to edit paths](#)

Source-only anchor: include/foundation/store.hpp:12

Why this implementation is on an edit path

[core-operation / N4: 4. Call Store::update \(REUSE\)](#)

Why: update validates text first, returns false for a missing ID when the text is valid, and prepares a replacement string before swapping it into place. The record ID stays unchanged.

Captured context excerpt

Bounded source context: lines 8-24 of 45 for anchors L12. 7 preceding and 21 following file lines are outside this excerpt. No containing callable/type was indexed for these anchors.

SHA-256 9fbe158ca20da19f21c0c69b79ec72a3ab559e3b91931f854b955a0c55e50e68 | Snapshot 2026-10-04T23:16:35-05:00

```
8  #include <vector>
9
10 namespace foundation {
11
12 using RecordId = std::uint64_t;
13
14 struct Record {
15     RecordId id;
16     std::string text;
17     bool operator==(const Record&) const = default;
18 };
19
20 // An owning, bounded collection. Callers serialize access to each instance.
21 // IDs never change or get reused. Returned values cannot mutate stored state.
22 class Store {
23 public:
24     static constexpr std::size_t max_text_bytes = 256;
```

[Complete source in explorer](#) | [Up to walkthrough index](#) | [Up to edit paths](#)